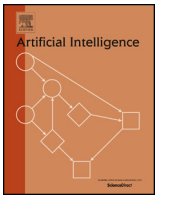
Artificial Intelligence 323 (2023) 103989

Contents lists available at ScienceDirect

Artificial Intelligence

journal homepage: www.elsevier.com/locate/artint



Learning reward machines: A study in partially observable reinforcement learning ☆

Rodrigo Toro Icarte [a,f,*], Toryn Q. Klassen [c,d], Richard Valenzano [e], Margarita P. Castro [b,f], Ethan Waldie [c], Sheila A. McIlraith [c,d]

[a] Department of Computer Science, Pontificia Universidad Católica de Chile, Santiago, RM, Chile
[b] Department of Industrial and Systems Engineering, Pontificia Universidad Católica de Chile, Santiago, RM, Chile
[c] Department of Computer Science, University of Toronto, Toronto, ON, Canada
[d] Vector Institute for Artificial Intelligence, Toronto, ON, Canada
[e] Toronto Metropolitan University, Toronto, ON, Canada
[f] Centro Nacional de Inteligencia Artificial CENIA, Santiago, RM, Chile

ARTICLE INFO

Article history:
Received 6 December 2021
Received in revised form 25 July 2023
Accepted 26 July 2023
Available online 1 August 2023

Keywords:
Reinforcement learning
Reward machines
Partial observability
Automata learning
Abstractions
Non-Markovian environments

ABSTRACT

Reinforcement Learning (RL) is a machine learning paradigm wherein an artificial agent interacts with an environment with the purpose of learning behaviour that maximizes the expected cumulative reward it receives from the environment. Reward machines (RMs) provide a structured, automata-based representation of a reward function that enables an RL agent to decompose an RL problem into structured subproblems that can be efficiently learned via off-policy learning. Here we show that RMs can be learned from experience, instead of being specified by the user, and that the resulting problem decomposition can be used to effectively solve partially observable RL problems. We pose the task of learning RMs as a discrete optimization problem where the objective is to find an RM that decomposes the problem into a set of subproblems such that the combination of their optimal memoryless policies is an optimal policy for the original problem. We show the effectiveness of this approach on three partially observable domains, where it significantly outperforms A3C, PPO, and ACER, and discuss its advantages, limitations, and broader potential.[1]

1. Introduction

A fundamental component of human intelligence is our ability to make decisions – to decide how to act. Indeed, decision making is essential not only to individuals, but to companies, to governments, and to computer-controlled systems that ensure the safe and effective operation of much of our modern infrastructure. Unfortunately, making good decisions can be hard. It can depend on complex inter-relationships between diverse factors, not all of them observable, or well understood. *Reinforcement learning (RL)* endeavours to solve sequential decision-making problems using minimal supervision and minimal

☆ This work is an extended version of our previous NeurIPS19 publication [58].

* Corresponding author.
E-mail address: rntoro@uc.cl (R. Toro Icarte).

[1] Our code is available at https://bitbucket.org/RToroIcarte/lrm.

https://doi.org/10.1016/j.artint.2023.103989
0004-3702/

prior knowledge. Its goal is to define artificial agents that learn optimal behaviour by interacting with an environment, which may take the form of a simulator or the real world [54]. Every interaction with the environment delivers a reward signal. An RL agent seeks to learn a *policy* (a mapping from observations to actions) that maximizes its expected cumulative reward, improving its policy over time by learning from its past experiences.

The use of neural networks for function approximation has led to many recent advances in RL. Such deep RL methods have allowed agents to learn effective policies in many complex environment including board games [52], video games [41], and robotic systems [2]. However, RL methods (including deep RL methods) often struggle when the environment is *partially observable*. Indeed, partial observability is one of the main challenges to solving real-world problems with RL [15,14]. This is because agents in such environments usually require some form of memory to learn optimal behaviour [53]. Recent approaches for giving memory to an RL agent either rely on recurrent neural networks (e.g., [24,40,29,60,50]), memory-augmented neural networks (e.g., [43,33,26]), or external memories that the agent can control using primitive actions (e.g., [35,45,65,57]).

In this work, we show that *reward machines (RMs)* are another useful tool for providing memory in a partially observable environment. RMs were originally conceived to provide a structured, automata-based representation of a reward function [55,56,7,10]. Exposed structure can be exploited by the *Q-learning for reward machines (QRM)* algorithm [55], which simultaneously learns a separate policy for each state in the RM. QRM has been shown to outperform standard and hierarchical deep RL over a variety of discrete and continuous domains. However, QRM was only defined for fully observable environments. Furthermore, the RMs were handcrafted by a user and then given to the RL agent, thus allowing the agent to exploit the exposed problem substructure. Here, we propose a method for learning an RM directly from experience in a partially observable environment, in a manner that allows the RM to serve as memory for an RL algorithm.

There are three main contributions of this work. The first is to propose a discrete optimization problem for learning reward machines from experience in a partially observable environment, where the objective is to find a reward machine that makes the problem *as Markovian as possible*. A requirement is that the RM learning method be given a finite set of detectors for properties that serve as the vocabulary for the RM. The model is also fed with traces collected by the agent while exploring the environment. Then, the optimization problem's objective function ranks the reward machines according to how well they predict future properties of the environment given the current RM state and observation.

Our second contribution is to study different methodologies to solve the resulting optimization problem for learning reward machines. In particular, we propose a *mixed integer linear programming (MILP)* model, a *constraint programming (CP)* model, and two *local search (LS)* methods. In our experiments, the best performance was obtained by the local search methods.

Finally, we show how to integrate our models for learning reward machines into the agent-environment interaction loop and show the effectiveness of doing so. By simultaneously learning an RM and a policy for the environment, we are able to significantly outperform several deep RL baselines that use recurrent neural networks as memory in three partially observable domains. We also extend the RM-tailored algorithm *Q-learning for reward machines (QRM)* to the case of partial observability where we see further gains when combined with our RM learning method.

This paper builds upon Toro Icarte et al. [58] – where we originally proposed to formulate the problem of learning an RM as a discrete optimization problem and solved it using tabu search. In this work, we provide further details about this learning pipeline and propose three novel formulations to learn reward machines. These new formulations include a MILP, CP, and LS model. We compare the performance of these models relative to tabu search and found that a local search approach with restarts is consistently better at finding high-quality RMs than tabu search. As a result, the performance of our method improves considerably with respect to the performance reported in our previous publication.

2. Preliminaries

RL agents learn policies from experience. When the problem is fully-observable, the underlying environment model is typically assumed to be a *Markov decision process (MDP)*. An MDP is a tuple $\mathcal{M} = \langle S, A, r, p, \gamma, \mu \rangle$, where S is a finite set of *states*, A is a finite set of *actions*, $r : S \times A \to \mathbb{R}$ is the *reward function*, $p(s, a, s')$ is the *transition probability distribution*, γ is the *discount factor*, and μ is the *initial state distribution* where $\mu(s_0)$ is the probability that the agent starts in state $s_0 \in S$. In addition, a subset of the states might be labelled as *terminal states*.

At the beginning of an *episode*, the environment is set to an initial state s_0, sampled from μ. Then, at time step t, the agent observes the current state $s_t \in S$ and executes an action $a_t \in A$. In response, the environment returns the next state $s_{t+1} \sim p(\cdot|s_t, a_t)$ and the immediate reward $r_{t+1} = r(s_t, a_t, s_{t+1})$. The process then repeats from s_{t+1} until potentially reaching a terminal state, when a new episode will begin.

The agent's goal is to collect as much reward from the environment as possible. To do so, it learns a *policy* $\pi(a|s)$, which is a probability distribution over the actions $a \in A$ given a state $s \in S$. As the agent interacts with the environment, it also improves its policy until (ideally) finding an *optimal policy* π^*. An optimal policy is a policy that maximizes the expected return received by the agent, which is formally defined as follows:

$$\pi^* = \arg\max_{\pi} \sum_{s \in S} \mu(s) \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \middle| s_0 = s \right] \tag{1}$$

Q-learning [61] is a well-known RL algorithm that uses samples of experience of the form (s_t, a_t, r_t, s_{t+1}) to estimate the optimal Q-function $q^*(s,a)$. Here, $q^*(s,a)$ is the expected return of selecting action a in state s and following an optimal policy π^* thereafter. During execution, Q-learning maintains the current estimate of the optimal Q-function (i.e., *Q-value*) for each state s and action a, $Q(s,a)$. Given a sampled experience (s_t, a_t, r_t, s_{t+1}), where s_{t+1} is the state reached after executing action a_t in state s_t and receiving a reward r_t, Q-learning updates $Q(s_t, a_t)$ towards $r_t + \gamma \max_{a' \in A} Q(s_{t+1}, a')$. Given enough experience, the Q-value estimates will converge to the optimal Q-function, and so an optimal policy π^* can be computed by always selecting the action a with the highest value of $Q(s,a)$ for each state $s \in S$.

Unfortunately, Q-learning is impractical when solving problems with large state spaces. In such cases, *function approximation* methods are often used. Instead of storing a Q-value for each state-action pair in a table, deep RL methods like DQN [41] and DDQN [59] represent the Q-function as $Q_\theta(s,a)$, where Q_θ is a neural network whose inputs are features of the state and the outputs are the Q-value estimates for each action $a \in A$. To train the network, mini-batches of experiences (s, a, r, s') are randomly sampled from an *experience replay* buffer and used to minimize the Bellman error. In the case of DQN, this is accomplished by minimizing the square error between $Q_\theta(s,a)$ and the Bellman estimate $r + \gamma \max_{a'} Q_{\theta'}(s', a')$. Note that the updates are made with respect to a *target network* with parameters θ'. The parameters θ' are held fixed when minimizing the square error, but updated to θ after a certain number of training updates. The role of the target network is to stabilize learning. DDQN follows a similar approach, but the Bellman estimate is computed by selecting the next action a' using Q_θ instead of the target network. This is, $r + \gamma Q_{\theta'}(s', \arg\max_{a'} Q_\theta(s', a'))$.

In partially observable problems, the underlying environment model is typically assumed to be a *Partially Observable Markov Decision Process (POMDP)*. A POMDP is a tuple $\mathcal{P}_\mathcal{O} = \langle S, O, A, r, p, \omega, \gamma, \mu \rangle$, where S, A, r, p, γ, and μ are defined as in an MDP, O is a finite set of *observations*, and $\omega(o|s)$ is the *observation probability distribution*. Interacting with a POMDP is similar to interacting with an MDP. The environment starts from a sampled initial state $s_0 \sim \mu$. At time step t, the agent is in state $s_t \in S$, executes an action $a_t \in A$, receives an immediate reward $r_{t+1} = r(s_t, a_t, s_{t+1})$, and moves to s_{t+1} according to $p(s_{t+1}|s_t, a_t)$. However, the agent does not observe s_t directly. Instead, the agent observes $o_t \in O$, which is linked to s_t via ω, where $\omega(o_t|s_t)$ is the probability of observing o_t from state s_t [8].

RL methods cannot be immediately applied to POMDPs because the transition probabilities and reward function are not necessarily Markovian w.r.t. O (though by definition they are w.r.t. S). As such, optimal policies may need to consider the complete history $(o_0, a_0, \ldots, a_{t-1}, o_t)$ of observations and actions when selecting the next action.[1] Several partially observable RL methods use a recurrent neural network to compactly represent the history, and then use a policy gradient method to train it. However, when we do have access to a full POMDP model $\mathcal{P}_\mathcal{O}$, then the history can be summarized into a *belief state*. A belief state is a probability distribution $b_t : S \to [0,1]$ over S, such that $b_t(s)$ is the probability that the agent is in state $s \in S$ given the history up to time t. The initial belief state is computed using the initial observation o_0: $b_0(s) \propto \omega(s, o_0)$ for all $s \in S$. The belief state b_{t+1} is then determined from the previous belief state b_t, the executed action a_t, and the resulting observation o_{t+1} as follows:

$$b_{t+1}(s') \propto \omega(s', o_{t+1}) \sum_{s \in S} p(s, a_t, s') b_t(s) \quad \text{for all } s' \in S. \tag{2}$$

Since the state transitions and reward function are Markovian w.r.t. b_t, the set of all belief states B can be used to construct the belief MDP $\mathcal{M}_B$, where the states of $\mathcal{M}_B$ are B, A are the actions, the transition probabilities are computed using equation (2), and the reward function r_b is as follows:

$$r_b(b, a) = \sum_{s \in S} r(s, a) b(s). \tag{3}$$

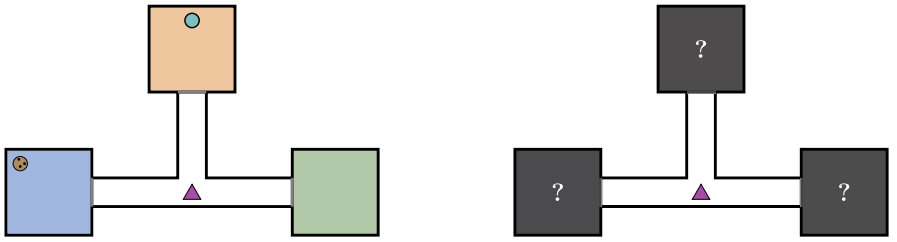
Any optimal policies for $\mathcal{M}_B$ is also optimal for the POMDP [8].

3. Reward machines for partially observable environments

In this section, we define RMs for the case of partial observability. We use the following problem as a running example to help explain various concepts.

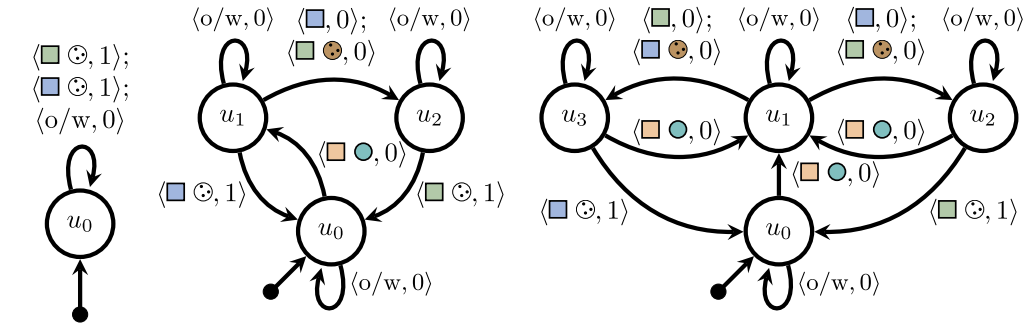
Example 1 *(The cookie domain).* The *cookie domain*, shown in Fig. 1a, has three rooms connected by a hallway. The agent (purple triangle) can move in the four cardinal directions. There is a button in the orange room that, when pressed, causes a cookie to randomly appear in either the green or the blue room. The agent receives a reward of +1 for reaching (and thus eating) the cookie and may then go and press the button again. Pressing the button before reaching a cookie will remove the existing cookie and cause a new cookie to randomly appear. There is no cookie at the beginning of the episode. This domain is partially observable since the agent can only see what is in the room that it currently occupies, as shown in Fig. 1b.

[1] Technically, the history of interactions should also include the immediate rewards [28], this is $h_t = (o_0, a_0, r_1, \ldots, a_{t-1}, r_t, o_t)$. However, we can remove the immediate rewards from the history without loss of generality because it is always possible to include the immediate reward r_t as part of the observation o_t.



(a) Cookie domain. (b) Agent's view.

Fig. 1. In the cookie domain, the agent can only see what is in the current room. (For interpretation of the colours in the figure(s), the reader is referred to the web version of this article.)



(a) Naive RM. (b) "Optimal" RM. (c) Perfect RM.

Fig. 2. Three possible Reward Machines for the Cookie domain.

RMs are finite state machines that are used to encode a reward function [55,56]. They are defined over a set of propositional symbols $\mathcal{P}$ that correspond to a set of high-level features that the agent can detect using a *labelling function* $L : O_\emptyset \times A_\emptyset \times O \to 2^{\mathcal{P}}$ where (for any set X) $X_\emptyset \triangleq X \cup \{\emptyset\}$. L assigns truth values to symbols in $\mathcal{P}$ given an environment experience $e = (o, a, o')$ where o' is the observation seen after executing action a when observing o. We use $L(\emptyset, \emptyset, o)$ to assign truth values to the initial observation. We call a truth value assignment of $\mathcal{P}$ a *high-level observation* because it provides a high-level view of the low-level environment observations via the labelling function L. A formal definition of an RM follows:

Definition 2 *(reward machine).* Given a set of propositional symbols $\mathcal{P}$, a reward machine is a tuple $\mathcal{R}_\mathcal{P} = \langle U, u_0, \delta_u, \delta_r \rangle$ where U is a finite set of states, $u_0 \in U$ is an initial state, δ_u is the state-transition function, $\delta_u : U \times 2^{\mathcal{P}} \to U$, and δ_r is the reward-transition function, $\delta_r : U \times 2^{\mathcal{P}} \to \mathbb{R}$.

One way of thinking about RMs is that they decompose problems into a set of high-level states U and define transitions using if-like conditions defined by δ_u. These conditions are over a set of binary properties $\mathcal{P}$ that the agent can detect using L. For example, in the cookie domain, $\mathcal{P} = \{$⚉, ☺, ●, □, □, □, □$\}$. These properties are true in the following situations: □, □, □, or □ is true if the agent is in a room of that colour; ⚉ is true if the agent is in the same room as a cookie; ● is true if the agent pushed the button with its last action; and ☺ is true if the agent ate a cookie with its last action.

Fig. 2 shows three possible reward machines for the cookie domain. We note that these three machines define the same reward signal, 1 for eating a cookie and 0 otherwise, but differ in their states and transitions. As a result, they differ with respect to the amount of information about the current POMDP state that can be inferred from the RM state, as we will see below.

Each RM starts in the initial state u_0. Edge labels in the figures provide a visual representation of the functions δ_u and δ_r. For example, label ⟨□ ☺, 1⟩ between state u_2 and u_0 in Fig. 2b represents $\delta_u(u_2, \{$□, ☺$\}) = u_0$ and $\delta_r(u_2, \{$□, ☺$\}) = 1$. Intuitively, this means that if the RM is in state u_2 and the agent just ate a cookie ☺ in room □, then the agent will receive a reward of 1 and the RM will transition to u_0. Notice that any properties not listed in the label are false (e.g. ⚉ must be false to take the transition labelled ⟨□ ☺, 1⟩). We also use multiple labels separated by a semicolon (e.g., "⟨□, 0⟩; ⟨□ ⚉, 0⟩") to describe different conditions for transitioning between the RM states, each with their own associated reward. The label ⟨o/w, r⟩ ("o/w" for "otherwise") on an edge from u_i to u_j means that transition will be made (and reward r received) if none of the other transitions from u_i can be taken.

Let us illustrate the behaviour of an RM using the one shown in Fig. 2c. The RM will stay in u_0 until the agent presses the button (causing a cookie to appear), whereupon the RM moves to u_1. From u_1 the RM may move to u_2 or u_3 depending on whether the agent finds a cookie when it enters another room. Finally, the RM moves back to u_0 from u_2 (or u_3) when

the agent eats a cookie. Note that it is possible to associate meaning with being in RM states. In the example, u_0 means that there is no cookie available, u_1 means that there is a cookie in some room (either blue or green), u_2 means that the cookie is in the green room, and u_3 means that the cookie is in the blue room.

When learning a policy for a given RM, one simple technique is to learn a policy $\pi(a|o,u)$ that considers the current observation $o \in O$ and the current RM state $u \in U$ to select action $a \in A$. Interestingly, a partially observable problem might be non-Markovian over O, but Markovian over $O \times U$ for some RM $\mathcal{R}_\mathcal{P}$. This is the case for the cookie domain with the RM from Fig. 2c, for example.

Q-learning for RMs (QRM) is another way to learn a policy by exploiting a given RM [55]. QRM learns one Q-function Q_u (i.e., policy) per RM state $u \in U$. Then, given any sample experience, the RM can be used to emulate how much reward would have been received had the RM been in any one of its states. Formally, experience $e = (o, a, o')$ can be transformed into a valid experience $(\langle o, u\rangle, a, \langle o', u'\rangle, r)$ used for updating Q_u for each $u \in U$, where $u' = \delta_u(u, L(e))$ and $r = \delta_r(u, L(e))$. Hence, any off-policy learning method can take advantage of these "synthetically" generated experiences to update all subpolicies simultaneously.

When tabular Q-learning is used, QRM is guaranteed to converge to an optimal policy on fully-observable problems [55]. However, in a partially observable environment, an experience e might be more or less likely depending on the RM state that the agent was in when the experience was collected. For example, experience e might be possible in one RM state u_i but not in RM state u_j. Thus, updating the policy for u_j using e as QRM does, would introduce an unwanted bias to Q_{u_j}. We will discuss how to (partially) address this problem in Section 6.

4. Learning reward machines from traces

To learn RMs, our overall idea is to search for an RM that can be used as external memory by an agent for solving a partially-observable problem. As input, our method will take a set of high-level propositional symbols $\mathcal{P}$ and a labelling function L that can detect them. Then, the key question is what properties should such an RM have.

Three proposals naturally emerge from the literature. The first comes from the work on learning *finite state machines (FSMs)* [4,64,51], which suggests learning the smallest RM that correctly mimics the external reward signal given by the environment, as in Giantamidis and Tripakis' method for learning Moore machines [19]. Unfortunately, such approaches would learn RMs of limited utility, like the one in Fig. 2a. This naive RM correctly predicts reward in the cookie domain (i.e., $+1$ for eating a cookie, zero otherwise) but provides no memory in support of solving the task.

The second proposal comes from the literature on learning *finite state controllers (FSC)* [39] and on *model-free RL* methods [54]. This work suggests looking for the RM whose optimal policy receives the most reward. For instance, the RM from Fig. 2b is "optimal" in this sense. It decomposes the problem into three states. The optimal policy for u_0 goes directly to press the button, the optimal policy for u_1 goes to the blue room and eats the cookie if present, and the optimal policy for u_2 goes to the green room and eats the cookie. Together, these three policies give rise to an optimal policy for the complete problem. This is a desirable property for RMs, but requires computing optimal policies in order to compare the relative quality of RMs, which seems prohibitively expensive. However, we believe that finding ways to efficiently learn "optimal" RMs is a promising future work direction.

Finally, the third proposal comes from the literature on *predictive state representations (PSRs)* [36], *deterministic Markov models (DMMs)* [37], and *model-based RL* [32]. This line of research suggests learning the RM that remembers sufficient information about the history to make accurate Markovian predictions about the next observation. For instance, the cookie domain RM shown in Fig. 2c is *perfect* w.r.t. this criterion. Intuitively, every transition in the cookie environment is already Markovian except for transitioning from one room to another. Depending on different factors, when entering into the green room there could be a cookie there (or not). The perfect RM is able to encode such information using 4 states: when at u_0 the agent knows that there is no cookie, at u_1 the agent knows that there is a cookie in the blue or the green room, at u_2 the agent knows that there is a cookie in the green room, and at u_3 the agent knows that there is a cookie in the blue room. Since keeping track of more information will not result in better predictions, this RM is *perfect*. Below, we develop a theory about perfect RMs and describe an approach for learning them from experience.

4.1. Perfect reward machines: formal definition and properties

The key insight behind perfect RMs is to use their states U and transitions δ_u to keep track of relevant past information such that the partially observable environment $\mathcal{P}_\mathcal{O}$ becomes Markovian with respect to $O \times U$. This is ensured by the following definition.

Definition 3 *(perfect reward machine).* An RM $\mathcal{R}_\mathcal{P} = \langle U, u_0, \delta_u, \delta_r\rangle$ is considered perfect for a POMDP $\mathcal{P}_\mathcal{O} = \langle S, O, A, r, p, \omega, \gamma, \mu\rangle$ w.r.t. a labelling function L iff for every trace $(o_0, a_0, \ldots, o_t, a_t)$ generated by any policy over $\mathcal{P}_\mathcal{O}$, the following holds:

$$\Pr(o_{t+1}, r_t | o_0, a_0, \ldots, o_t, a_t) = \Pr(o_{t+1}, r_t | o_t, x_t, a_t),$$

where $x_0 = u_0$ and $x_t = \delta_u(x_{t-1}, L(o_{t-1}, a_{t-1}, o_t))$.[2]

Two important properties follow from Definition 3. First, if the set of reachable belief states B for the POMDP $\mathcal{P}_\mathcal{O}$ is finite, then there exists a perfect RM for $\mathcal{P}_\mathcal{O}$. Recall that a belief state models the probability of being at any POMDP state given the previous interactions with the environment. When the model of $\mathcal{P}_\mathcal{O}$ is known, we can compute the current belief state using the Bayes rule, as discussed in Section 2. This first property states that if there is a finite set of belief states reachable from the initial state, then there exists a perfect RM for that environment.

Theorem 4. *For any POMDP $\mathcal{P}_\mathcal{O}$ with a finite reachable belief space, there exists a perfect RM for $\mathcal{P}_\mathcal{O}$.*

Proof. If the reachable belief space B is finite, we can construct an RM that keeps track of the current belief state using one RM state per belief state and emulating their progression using δ_u, and one propositional symbol for every action-observation pair. Thus, the current belief state b_t can be inferred from the last observation, last action, and the current RM state. Hence, the equality from Definition 3 holds. □

The second property of perfect RMs is that their optimal policies are also optimal for the POMDP. This means that summarizing the history $h_t = (o_0, a_0, \ldots, a_{t-1}, o_t)$ as $\phi(h_t) = \langle u_t, o_t \rangle$, where u_t is the current RM state and o_t is the current observation, results in *globally* optimal policies – i.e., $\pi^*(a_t|h_t) = \pi^*(a_t|u_t, o_t)$.

Theorem 5. *Let $\mathcal{R}_\mathcal{P}$ be a perfect RM for a POMDP $\mathcal{P}_\mathcal{O}$, then any optimal policy for $\mathcal{R}_\mathcal{P}$ w.r.t. the environmental reward is also optimal for $\mathcal{P}_\mathcal{O}$.*

Proof. As the next observation and immediate reward probabilities can be predicted from $O \times U \times A$, a perfect RM for $\mathcal{P}_\mathcal{O}$ also models the belief MDP $\mathcal{M}_B$ for $\mathcal{P}_\mathcal{O}$. As such, optimal policies over $O \times U$ are equivalent to optimal policies over $\mathcal{M}_B$, which are optimal over $\mathcal{P}_\mathcal{O}$ [8]. □

4.2. Perfect reward machines: how to learn them

We now consider the problem of learning a perfect RM from traces, assuming one exists w.r.t. the given labelling function L. Recall that a perfect RM transforms the original problem into a Markovian problem over $O \times U$. Hence, we should prefer RMs that accurately predict the next observation o' and the immediate reward r from the current observation o, RM state u, and action a. This might be achieved by collecting a training set of traces from the environment, fitting a predictive model for $\Pr(o', r|o, u, a)$, and picking the RM that makes better predictions. However, this approach can be very expensive, especially considering that the observations might be images.

Instead, we propose an alternative that focuses on a necessary condition for a perfect RM: the RM must predict what is *possible* and *impossible* in the environment at the abstract level given by the labelling function. E.g., it is impossible to be at u_3 in the RM from Fig. 2c and make the high-level observation $\{\square, \cookie\}$, because the RM reaches u_3 only if the cookie was seen in the blue room (or not to be in the green room) and it leaves u_3 as soon as the agent eats the cookie (or presses the button).

This idea is formalized in the optimization model (LRM). Let $\mathcal{T} = \{\mathcal{T}_0, \ldots, \mathcal{T}_n\}$ be a set of traces, where each trace $\mathcal{T}_i$ is a sequence of observations, actions, and rewards:

$$\mathcal{T}_i = (o_{i,0}, a_{i,0}, r_{i,1}, \ldots, a_{i,t_i-1}, r_{i,t_i}, o_{i,t_i}). \tag{4}$$

We now look for an RM $\langle U, u_0, \delta_u, \delta_r \rangle$ that can be used to predict $L(e_{i,t+1})$ from $L(e_{i,t})$ and the current RM state $x_{i,t}$, where $e_{i,t+1}$ is the experience $(o_{i,t}, a_{i,t}, o_{i,t+1})$ and we define $e_{i,0}$ as $(\emptyset, \emptyset, o_{i,0})$. The model parameters are the set of traces $\mathcal{T}$, the set of propositional symbols $\mathcal{P}$, the labelling function L, and a maximum number of states in the RM $u_{\max}$. The model also uses the sets $I = \{0 \ldots n\}$, $T_i = \{0 \ldots t_i - 1\}$, and Σ, where I contains the indices of the traces, T_i their time steps, and Σ contains all the high-level observations that appear in $\mathcal{T}$. The model has two auxiliary variables $x_{i,t}$ and $N_{u,\sigma}$. Variable $x_{i,t} \in U$ represents the state of the RM after observing trace $\mathcal{T}_i$ up to time t. Variable $N_{u,\sigma} \subseteq \Sigma$ is the set of all the next high-level observations seen from the RM state u and the high-level observations σ in $\mathcal{T}$. In other words, $\sigma' \in N_{u,\sigma}$ iff $u = x_{i,t}$, $\sigma = L(e_{i,t})$, and $\sigma' = L(e_{i,t+1})$ for some trace $\mathcal{T}_i$ and time t.

$$\underset{\langle U, u_0, \delta_u, \delta_r \rangle}{\text{minimize}} \sum_{i \in I} \sum_{t \in T_i} \log(|N_{x_{i,t}, L(e_{i,t})}|) \tag{LRM}$$

$$s.t.\ \langle U, u_0, \delta_u, \delta_r \rangle \in \boldsymbol{\mathcal{R}}_\mathcal{P} \tag{5}$$

[2] Note that x_t is the RM state that the agent is in at time t. We will make use of this notation below.

$$|U| \leq u_{\max} \tag{6}$$

$$x_{i,t} \in U \qquad \forall i \in I, t \in T_i \cup \{t_i\} \tag{7}$$

$$x_{i,0} = u_0 \qquad \forall i \in I \tag{8}$$

$$x_{i,t+1} = \delta_u(x_{i,t}, L(e_{i,t+1})) \qquad \forall i \in I, t \in T_i \tag{9}$$

$$N_{u,\sigma} \subseteq \Sigma \qquad \forall u \in U, \sigma \in \Sigma \tag{10}$$

$$L(e_{i,t+1}) \in N_{x_{i,t},L(e_{i,t})} \qquad \forall i \in I, t \in T_i \tag{11}$$

Constraints (5) and (6) ensure that we find a well-formed RM over $\mathcal{P}$ with at most $u_{\max}$ states, where $\mathcal{R}_\mathcal{P}$ is the set of all possible RMs. Constraints (7), (8), and (9) ensure that $x_{i,t}$ is equal to the current state of the RM, starting from u_0 and following δ_u. Constraints (10) and (11) ensure that the sets $N_{u,\sigma}$ contain every $L(e_{i,t+1})$ that has been seen right after σ and u in $\mathcal{T}$. The objective function comes from maximizing the log-likelihood for predicting $L(e_{i,t+1})$ using a uniform distribution over all the possible options given by $N_{u,\sigma}$.

A key property of this formulation is that any perfect RM is optimal w.r.t. the objective function in (LRM) when the number of traces tends to infinity.

Theorem 6. *When the set of training traces (and their lengths) tends to infinity and is collected by a policy such that $\pi(a|o) > \epsilon$ for all $o \in O$ and $a \in A$, any perfect RM with respect to L and at most u_{max} states will be an optimal solution to the formulation (LRM).*

Proof. In the limit, $\sigma' \in N_{u,\sigma}$ if and only if the probability of observing σ' after executing an action from the RM state u while observing σ is non-zero. In particular, for all $i \in I$ and $t \in T$, the cardinality of $N_{x_{i,t},L(e_{i,t})}$ will be minimal for a perfect RM. This property follows from the fact that perfect RMs make perfect predictions for the next observation o' given o, u, and a. Therefore, as we minimize the sum over $\log(|N_{x_{i,t},L(e_{i,t})}|)$, the objective value for any perfect RM must be minimal. □

Finally, note that the definition of a perfect RM does not impose conditions over the rewards associated with the RM (i.e., δ_r). This is why δ_r is a free variable in the model (LRM). However, in order to apply methods that exploit RM structure (such as QRM), we still need δ_r to model the external reward signals given by the environment. To do so, we estimate $\delta_r(u, \sigma)$ using its empirical expectation over $\mathcal{T}$ – as commonly done when constructing belief MDPs [8]. Formally,

$$\delta_r(u, \sigma) = \frac{\sum_{i \in I, t \in T_i} r_{i,t+1} 1_{u = x_{i,t} \wedge \sigma = L(e_{t+1})}}{\sum_{i \in I, t \in T_i} 1_{u = x_{i,t} \wedge \sigma = L(e_{t+1})} + \epsilon} \quad \text{for all } u \in U \text{ and } l \in 2^\mathcal{P}, \tag{12}$$

where $1_c = 1$ if condition c holds (zero otherwise) and ϵ is a small constant.

5. Searching for a perfect reward machine

We now describe different approaches to solve (LRM). These include a *mixed integer linear programming (MILP)* model, a *constraint programming (CP)* model, and two *local search (LS)* models. All our models are guaranteed to find optimal solutions given sufficient resources. But first, let us introduce two preprocessing steps over the training traces that our models use: *trace compression* and *prefix trees (PTs)*.

5.1. Trace compression

Recall that (LRM) works over the high-level observations σ_t given by the labelling function $L(e_t)$, where e_t represents the experience (o_t, a_t, o_{t+1}) at time t. Thus, the first step is to transform each trace $\mathcal{T}_i$ from being a sequence of interactions with the environment into a sequence of truth value assignments of $\mathcal{P}$ via L:

$$\mathcal{T}_i = (o_{i,0}, a_{i,0}, r_{i,1}, \ldots, a_{i,t_i-1}, r_{i,t_i}, o_{i,t_i}) \xrightarrow{\sigma_t = L(e_t)} \tau_i = (\sigma_{i,0}, \sigma_{i,1}, \ldots, \sigma_{i,t_i}).$$

In the abstract space given by $\mathcal{P}$ and L, it is usually the case that the same high-level observation is seen many times in a row. For instance, a typical high-level trace in the cookie domain would look as follows:

$\tau = (\square, \square, \square, \square, \square, \square, \square\bullet, \square, \square, \square, \square, \square, \square, \square, \square, \square, \square, \blacksquare, \square, \square, \square, \square, \square, \square, \square, \square, \square, \square\bullet)$.

This trace indicates that the agent was first at the hallway ($\square$), and then it moved to the orange room ($\square$) and pressed the button ($\square\bullet$). After that, it returned to the hallway ($\square$), noticed that there was no cookie in the blue room ($\blacksquare$), came back to the hallway ($\square$), and finally found a cookie in the green room ($\square\bullet$). Let us assume that the trace ended there for simplicity. As you can see, the most informative moments are when the high-level observations change. Indeed, knowing that the agent stayed at the hallway during 4, 7, or 8 steps is not particularly useful in this case. This suggests that we could

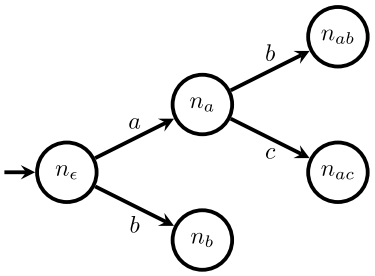


Fig. 3. A PT for $\{(b), (ab), (ac)\}$. We took this example from [19].

potentially compress the training trace (without losing relevant information) by removing duplicated high-level observations that appear consecutively in the trace. For instance, the previous trace will look as follows after being compressed:

$$\tau_c = (\square, \blacksquare, \blacksquare\bullet, \blacksquare, \square, \blacksquare, \square, \blacksquare\otimes).$$

Compressing the traces is an optional preprocessing step that has advantages and disadvantages. The advantage is that it considerably improves the quality of the RMs that the models find given a fixed computational budget. Intuitively, any model has to go over the traces to evaluate how good an RM is. Compressing the traces reduces the computational cost of doing so. The disadvantage is that, by compressing the traces, we are assuming that observing two or more times the same high-level observation consecutively does not provide further information about the current POMDP state. If that is the case, then we can compress the traces and the model will still find optimal solutions for (LRM). If that is not the case, then compressing the traces might help the models to find high-quality RMs faster but they will not find optimal solutions.

Two inconsistencies might arise if we compress the traces. First, note that we are learning RMs using compressed traces but then testing them over uncompressed traces. This is a problem because the optimal solution for (LRM) will exploit the fact that no training trace has the same high-level observation twice in a row. However, when the agent uses the learned RM in the environment (i.e., at test time), it might encounter the same high-level observation many times in a row and, thus, update the RM state in ways that were unintended by (LRM). For that reason, if we do compress the traces, we have to include the following additional constraint to (LRM):

$$\left[\delta_u(u', \sigma) = u\right] \Rightarrow \left[\delta_u(u, \sigma) = u\right] \qquad \forall u, u' \in U, \sigma \in \Sigma \tag{13}$$

This constraint ensures that the learned reward machine will not enter and leave a state u using the same high-level observation σ.

The second inconsistency relates to constraint (9). According to that constraint, the second high-level observation of the trace is used to progress the initial state in the reward machine (and not the first one). Indeed, (LRM) dictates that $x_0 = u_0$ and $x_1 = \delta_u(u_0, \blacksquare)$ for τ_c. However, if we were processing the uncompressed version of τ_c (i.e., τ), then the initial RM state would have been updated using $x_1 = \delta_u(u_0, \square)$ instead of $x_1 = \delta_u(u_0, \blacksquare)$ because $\square$ appears many times in a row at the beginning of τ. We can solve this inconsistency by not compressing the first high-level observation of the trace. This is, to always include the first high-level observation and only compress from the second high-level observation forward. In our example, the proper manner of compressing τ would be as follows:

$$\tau'_c = (\square, \square, \blacksquare, \blacksquare\bullet, \blacksquare, \square, \blacksquare, \square, \blacksquare\otimes).$$

5.2. Prefix trees (PTs)

Regardless of whether we compress or not the training traces, from now on we will be working with a set of $n+1$ traces $\mathcal{T} = \{\tau_0, \ldots, \tau_n\}$, where each trace is composed of a sequence of high-level observations: $\tau_i = (\sigma_{i,0}, \ldots, \sigma_{i,t_i})$. Then, (LRM) defines an independent variable $x_{i,t}$ which models the current RM state at time step t given the trace τ_i for all $i \in I$ and $t \in T_i$. However, doing so does not exploit the fact that there exist large sets of $x_{i,t}$ variables whose values are equivalent for any reward machine. These are cases where the prefix of two traces τ_i and τ_j are identical up to some time step t. Indeed, we know that variables $x_{i,t} = x_{j,t}$ if $\sigma_{i,t'} = \sigma_{j,t'}$ for all $t' < t$ because the transitions of a reward machine are deterministic. We can compactly capture this information using *prefix trees (PTs)* [25].

PTs merge all the training traces into one large tree, where each trace becomes a branch in this tree. As an example, Fig. 3 shows the PT for the following set of traces: $\tau_1 = (b)$, $\tau_2 = (a, a)$, and $\tau_3 = (a, b)$. Each node in a PT represents a prefix that appears in one or more training traces. In the example, there are 5 nodes corresponding to the 5 possible prefixes in $\{\tau_1, \tau_2, \tau_3\}$: ϵ, (a), (b), (a, b), and (a, c), where ϵ represents the empty trace. Thus, instead of assigning RM states to each variable $x_{i,t}$, our models assign RM states to each node in the PT. That way, these models are forced to assign the same sequence of RM states to all traces as long as their prefixes are identical. For instance, assigning an RM state to node n_a in

Algorithm 1 Converting Training Data into Prefix Trees.

```
1: Input: {τ_0, ..., τ_n}
2: root_node ← create_root_node()
3: for i = 0 to n do
4:     node ← root_node
5:     for t = 0 to |τ_i| − 1 do
6:         node.increase_trace_counter_by_one()
7:         if not node.has_child(σ_{i,t}) then
8:             node.add_child(σ_{i,t})
9:         node ← node.get_child(σ_{i,t})
10: return root_node
```

the PT will automatically assign the same RM state to $x_{2,1}$ and $x_{3,1}$ because n_a represents the current RM state after any trace observes a at the initial time step and, in this case, 'a' is the first observation of τ_2 and τ_3.

Algorithm 1 shows the pseudo-code to constructing a PT from a given set of traces $\{\tau_0, \ldots, \tau_n\}$. Starting at the root node, the code goes over all the training traces and adds them as branches of the tree (lines 7-8). We also count how many training traces pass through each node in the tree (line 6). This information helps us defining the right weights to penalize predictions in the objective function when reformulating (LRM) from assigning RM states to time steps to assigning RM states to nodes in the PT.

5.3. Solving LRM via mixed integer linear programming (MILP)

In this section, we present a MILP model for (LRM). MILP solvers are able to solve optimization problems with linear constraints and objective functions. They can also handle continuous and discrete variables. MILP solvers are the state of the art for solving a wide range of discrete optimization problems and they are guaranteed to find optimal solutions given sufficient resources [31].

Our MILP model receives as input $u_{\max}$ and the PT built using the training traces $\mathcal{T} = \{\tau_0, \ldots, \tau_n\}$. Note that the traces in $\mathcal{T}$ might or might not be compressed. We use the following notation to refer to the different elements in the PT: n_{root} is its root node, S is a set containing all the nodes in PT except for n_{root}, $S_{\text{in}} \subset S$ is a subset of S that only contains the inner nodes of PT, w_n is the number of training traces that pass through node n, $p(n)$ is the parent of node n, $o(n)$ is the high-level observation associated with the edge between nodes $p(n)$ and n, and $C(n)$ is the set of children of node n. This model assumes that the set Σ contains all the high-level observations that appear in $\mathcal{T}$, $|U| = u_{\max}$, and $K = 2^{|\Sigma|}$.

The idea behind our MILP model is to assign RM states to each node in the PT. Then, we look for an assignment that (i) can be produced by a deterministic machine and (ii) optimizes the same objective as (LRM). To achieve this, we use the following decision variables. Variable $x_{n,u} \in \{0, 1\}$ indicates if node $n \in S \cup \{n_{\text{root}}\}$ is assigned the RM state $u \in U$. Variable $d_{u,\sigma,u'} \in \{0, 1\}$ represents the possible transition from state $u \in U$ to state $u' \in U$ given observation $\sigma \in \Sigma$ in the RM. Formally, this means that $d_{u,\sigma,u'} = 1$ iff $\delta_u(u, \sigma) = u'$. Variable $p_{u,\sigma,\sigma'} \in \{0, 1\}$ indicates if $\sigma' \in \Sigma$ is a possible next observation at RM state $u \in U$ when observing $\sigma \in \Sigma$, where $p_{u,\sigma,\sigma'} = 1$ iff $\sigma' \in N_{u,\sigma}$. Variable $y_{u,\sigma,m} \in \{0, 1\}$ represents the cardinality of $N_{u,\sigma}$, meaning that $y_{u,\sigma,m} = 1$ iff $|N_{u,\sigma}| = m$. Lastly, variables z_n represent the log-likelihood cost for the predictions associated with node $n \in S$. The full model is then as follows:

$$\min \sum_{n \in S} w_n \cdot z_n \tag{MILP}$$

$$s.t.\ z_n \geq \sum_{m=1}^{K} y_{u,\sigma,m} \log(m) - (1 - x_{n,u}) \log(K) \qquad \forall n \in S, u \in U, \sigma = o(n) \tag{14}$$

$$\sum_{m=1}^{K} y_{u,\sigma,m} = 1 \qquad \forall u \in U, \sigma \in \Sigma \tag{15}$$

$$\sum_{\sigma' \in \Sigma} p_{u,\sigma,\sigma'} = \sum_{m=1}^{K} y_{u,\sigma,m} \cdot m \qquad \forall u \in U, \sigma \in \Sigma \tag{16}$$

$$p_{u,o(n),o(n')} \geq x_{n,u} \qquad \forall u \in U, n \in S_{\text{in}}, n' \in C(n) \tag{17}$$

$$\sum_{u' \in U} d_{u,\sigma,u'} = 1 \qquad \forall u \in U, \sigma \in \Sigma \tag{18}$$

$$\sum_{u \in U} x_{n,u} = 1 \qquad \forall n \in S, u \in U \tag{19}$$

$$x_{n,u_0} = 1 \qquad n = n_{\text{root}} \tag{20}$$

$$x_{p(n),u} + x_{n,u'} - 1 \leq d_{u,o(n),u'} \qquad \forall u, u' \in U, n \in S \tag{21}$$

$$d_{u,\sigma,u'} \leq d_{u',\sigma,u'} \qquad \forall u, u' \in U, u \neq u', \sigma = \Sigma \tag{22}$$

$$x_{n,u} \in \{0,1\} \qquad \forall n \in S \cup \{n_{\text{root}}\}, u \in U \tag{23}$$

$$d_{u,\sigma,u'} \in \{0,1\} \qquad \forall u, u' \in U, \sigma \in \Sigma \tag{24}$$

$$p_{u,\sigma,\sigma'} \in \{0,1\} \qquad \forall u \in U, \sigma, \sigma' \in \Sigma \tag{25}$$

$$y_{u,\sigma,m} \in \{0,1\} \qquad \forall u \in U, \sigma \in \Sigma, m \in \{1..K\} \tag{26}$$

$$z_n \geq 0 \qquad \forall n \in S \tag{27}$$

The objective function of (MILP) is a sum over the prediction costs at each node in the tree weighted by how many traces pass through that node. Constraint (14) models the log-likelihood cost for each node in the tree. Constraints (15) and (16) compute the cardinality of $N_{u,l}$. Constraint (17) defines the possible predictions given an RM state and high-level observation. Constraint (18) enforces that for each RM state a high-level observation can lead to exactly one other RM state. Constraint (19) enforces that exactly one RM state is assigned to each node in the tree. Constraint (20) assigns the initial RM state to the root node, and constraint (21) enforces that there exists a deterministic δ_u that can produce the assignment of RM states to tree nodes. Constraints (23)-(27) correspond to the variables' domains. Finally, we note that constraint (22) is an optional constraint that should be included only if the training traces were compressed. This constraint enforces that the RM state does not change after observing the same high-level observation two times consecutively.

5.4. Solving LRM via constraint programming (CP)

CP is another technique for solving discrete optimization problems. CP is less restrictive than MILP in the type of variables and constraints that it can handle. For instance, our MILP model had to include many auxiliary decision variables (e.g., $x_{n,u}$, $p_{u,\sigma,\sigma'}$, $y_{u,\sigma,m}$, and z_n) in order to linearize different aspects of (LRM). In contrast, our CP model only uses one set of decision variables $d_{u,\sigma}$ to model $\delta_u(u,\sigma)$ and all other elements from (LRM) are defined w.r.t. those variables. In addition, CP solvers are also guaranteed to find optimal solutions given enough resources [49].

Our CP model receives the same inputs as our MILP model, including the PT constructed using the (potentially compressed) training traces $\mathcal{T}$. Recall that the different elements in the PT are referred as follows: n_{root} is the root node, S is the set containing all nodes but n_{root}, w_n is the number of training traces that pass through node n, $p(n)$ is the parent of node n, $o(n)$ is the high-level observation between $p(n)$ and n, and $C(n)$ is the set of children of node n. The model also uses the set $U = \{0...u_{\max} - 1\}$, the set Σ with all the high-level observations in $\mathcal{T}$, and the set $S(\sigma,\sigma')$ containing all the nodes where σ' is observed immediately after observing σ:

$$S(\sigma,\sigma') = \{n \mid n \in S, n' \in C(n), \sigma = o(n), \sigma' = o(n')\}.$$

As we previously mentioned, the only decision variables are $d_{u,\sigma} \in U$ for all $u \in U$ and $\sigma \in \Sigma$. Note that $d_{u,\sigma}$ is an integer variable that goes from 0 to $u_{\max} - 1$ and it models the output of $\delta_u(u,\sigma)$ – i.e., if the RM state is u and the agent observes σ, then the next RM state will be $d_{u,\sigma}$. With that, the complete model is as follows:

$$\min \sum_{n \in S} w_n \cdot \texttt{log}(y_{x_n,o(n)}) \tag{CP}$$

$$s.t.\ y_{u,\sigma} \doteq \sum_{\sigma' \in \Sigma} p_{u,\sigma,\sigma'} \qquad \forall u \in U, \sigma \in \Sigma \tag{28}$$

$$p_{u,\sigma,\sigma'} \doteq \texttt{logical\_or}\left(\{x_n = u | n \in S(\sigma,\sigma')\}\right) \qquad \forall u \in U, \sigma, \sigma' \in \Sigma \tag{29}$$

$$x_n \doteq d_{u,o(n)} \qquad \forall n \in S, u = x_{p(n)} \tag{30}$$

$$x_n \doteq 0 \qquad n = n_{\text{root}} \tag{31}$$

$$\texttt{if\_then}(d_{u,\sigma} = u', d_{u',\sigma} = u') \qquad \forall u, u' \in U, \sigma \in \Sigma \tag{32}$$

$$d_{u,\sigma} \in U \qquad \forall u \in U, \sigma \in \Sigma \tag{33}$$

This model uses the formalism and global constraints available in IBM ILOG CP Optimizer [27]. Its only decision variables are $d_{u,\sigma}$, defined in constraint (33). Note that the domain of $d_{u,\sigma} \in U$ forces the RM to be deterministic, since there is exactly one possible transition for each $u \in U$ and $\sigma \in \Sigma$. Using $d_{u,\sigma}$, the model defines three auxiliary *CP expressions* in equations (28)-(31). Expression $y_{u,\sigma}$ represents the cardinality of the prediction set $N_{u,\sigma}$ after observing $\sigma \in \Sigma$ from the RM state $u \in U$. Expression $p_{u,\sigma,\sigma'}$ is one if and only if it is possible to observe $\sigma' \in \Sigma$ from the RM state $u \in U$ after observing $\sigma \in \Sigma$ (and zero otherwise). This expression uses a logical OR constraint, $\texttt{logical\_or}(Z)$ which returns 1 iff at least one element $z \in Z$ is true. Expression $x_n \in U$ indicates the RM state assigned to the tree node n. Finally, the objective function is a weighted sum over the prediction errors and constraint (32) is an optional constraint used if the traces were compressed. This constraint enforces a self loop $\delta_u(u',\sigma) = u'$ if $\delta_u(u,\sigma) = u'$ for some $u \in U$.

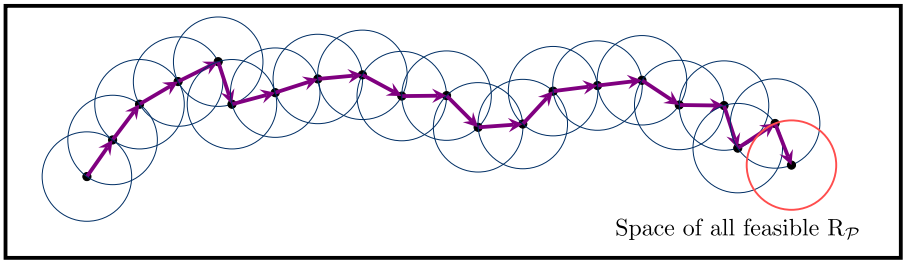


Fig. 4. Local search approaches start from some feasible solution and iteratively move to the best solution within its neighbourhood. This process repeats until reaching a locally optimal solution.

Algorithm 2 A local search approach with restarts to solve (LRM).

```
1: Input: Σ, 𝒯, u_max, t_max
2: t ← 0, c* ← ∞, ℛ* ← None
3: while t ≤ t_max do
4:     c_p ← ∞
5:     ℛ ← sample_a_reward_machine(𝒯, Σ, u_max)
6:     c ← evaluate_reward_machine(ℛ, 𝒯)
7:     if c < c* then
8:         c* ← c, ℛ* ← ℛ
9:     while t ≤ t_max and c < c_p do
10:        t ← t + 1, c_p ← c
11:        𝒩 ← get_neighbours(ℛ, Σ, u_max)
12:        for ℛ_n ∈ 𝒩 do
13:            c_n ← evaluate_reward_machine(ℛ_n, 𝒯)
14:            if c_n < c then
15:                c ← c_n, ℛ ← ℛ_n
16:        if c < c* then
17:            c* ← c, ℛ* ← ℛ
18: return ℛ*
```

5.5. Solving LRM via local search (LS) and tabu search (TS)

Finally, here we present our local search methods. We note that MILP and CP are known as *exact methods*. They incrementally construct a search tree where each branch represents a feasible solution to the problem and use different relaxation and propagation rules to prune this tree as much as possible. This approach allows them to find optimal solutions and prove that those solutions are optimal for small to medium size problems. However, they struggle when facing large problems because the size of the tree grows exponentially with the number of variables and computing relaxations and propagation rules become more expensive with the number of constraints.

When solving large scale problems, the best results are often obtained by *heuristic* methods. Heuristic methods propose polynomial-time approximations to solve NP-hard problems. They favour finding good solutions over providing strong optimality guarantees. Here, we explore two local search methods [1].

Fig. 4 shows how local search works. In the figure, each point inside the rectangle represents a feasible solution to the problem. Local search starts from a feasible solution and evaluates its objective function. Then, it evaluates the objective function of all the solutions near the current solution and then moves to the best solution within that region. This step is represented by a violet arrow in the figure. The process then repeats until a locally optimal solution is reached, where no neighbouring solution is better than the current solution.

Unfortunately, local search can converge to locally optimal solutions that might be far from a globally optimal solution. To deal with this issue, one option is to *restart* local search when it finds a locally optimal solution and start over from a different initial solution. Another option is to use *tabu search* [20]. Tabu search is a local search approach that saves the last n solutions in a *tabu list* and always moves to the best neighbour that is not in the list. This allows tabu search to escape locally optimal solutions. Here, we explore these two options for solving (LRM).

Algorithm 2 shows a local search approach with restarts to learn an RM. This algorithm receives the set of high-level observations Σ, the training traces $\mathcal{T}$, the maximum number of RM state $u_{\max}$, and some termination criteria such as a time limit or a maximum number of steps $t_{\max}$. The algorithm starts from a randomly generated RM (line 5). The initial RM is sampled from a uniform distribution. That is, every RM with at most $u_{\max}$ states can be selected with equal probability. On each iteration, the algorithm evaluates all neighbouring RMs (lines 9-17). We define the neighbourhood of an RM as the set of RMs that differ by exactly one transition (i.e., removing/adding a transition, or changing its value) and evaluate RMs using the objective function of (LRM). When all neighbouring RMs are evaluated, the algorithm moves to the neighbouring RM with the lowest objective value (line 15), and the process repeats. If at any point a locally optimal solution is reached, then the algorithm starts over from another randomly generated RM (line 9). Finally, the best RM seen so far is returned when the terminal condition is met (line 18).

Algorithm 3 A tabu search approach to solve (LRM).

1: **Input:** Σ, $\mathcal{T}$, $u_{\max}$, $t_{\max}$, τ_{size}
2: $\tau \leftarrow$ initialize_tabu_list(τ_{size})
3: $t \leftarrow 0$, $c^* \leftarrow \infty$, $\mathcal{R}^* \leftarrow$ None
4: **while** $t \leq t_{\max}$ **do**
5: $\quad \mathcal{R} \leftarrow$ sample_a_reward_machine($\mathcal{T}$, Σ, $u_{\max}$)
6: $\quad c \leftarrow$ evaluate_reward_machine($\mathcal{R}$, $\mathcal{T}$)
7: $\quad$ **if** $c < c^*$ **then**
8: $\quad\quad c^* \leftarrow c$, $\mathcal{R}^* \leftarrow \mathcal{R}$
9: $\quad$ **while** $t \leq t_{\max}$ **and** $\mathcal{R} \notin \tau$ **do**
10: $\quad\quad t \leftarrow t + 1$, $c \leftarrow \infty$
11: $\quad\quad \tau \leftarrow$ add_reward_machine_to_tabu_list(τ, $\mathcal{R}$)
12: $\quad\quad \mathcal{N} \leftarrow$ get_neighbours($\mathcal{R}$, Σ, $u_{\max}$)
13: $\quad\quad$ **for** $\mathcal{R}_n \in \mathcal{N} \setminus \tau$ **do**
14: $\quad\quad\quad c_n \leftarrow$ evaluate_reward_machine($\mathcal{R}_n$, $\mathcal{T}$)
15: $\quad\quad\quad$ **if** $c_n < c$ **then**
16: $\quad\quad\quad\quad c \leftarrow c_n$, $\mathcal{R} \leftarrow \mathcal{R}_n$
17: $\quad\quad$ **if** $c < c^*$ **then**
18: $\quad\quad\quad c^* \leftarrow c$, $\mathcal{R}^* \leftarrow \mathcal{R}$
19: **return** $\mathcal{R}^*$

Algorithm 4 Algorithm to simultaneously learn a reward machine and a policy.

1: **Input:** $\mathcal{P}$, L, $u_{\max}$, t_{w}
2: $\mathcal{T} \leftarrow$ collect_traces(t_{w})
3: $\mathcal{R}, N \leftarrow$ learn_rm($\mathcal{P}$, L, $\mathcal{T}$, $u_{\max}$)
4: $o \leftarrow$ env_get_initial_state(), $u \leftarrow u_0$, $\sigma \leftarrow L(\emptyset, \emptyset, o)$
5: $\pi \leftarrow$ initialize_policy()
6: **for** $t = 1$ **to** t_{train} **do**
7: $\quad a \leftarrow$ select_action(π, o, u)
8: $\quad o', r,$ done $\leftarrow$ env_execute_action(a)
9: $\quad u', \sigma' \leftarrow \delta_u(u, L(o, a, o')), L(o, a, o')$
10: $\quad \pi \leftarrow$ improve(π, o, u, σ, a, r, o', u', σ', done, N)
11: $\quad$ **if** $\sigma' \notin N_{u,\sigma}$ **then**
12: $\quad\quad \mathcal{T} \leftarrow \mathcal{T} \cup$ get_current_trace()
13: $\quad\quad \mathcal{R}', N \leftarrow$ relearn_rm($\mathcal{R}$, $\mathcal{P}$, L, $\mathcal{T}$, $u_{\max}$)
14: $\quad\quad$ **if** $\mathcal{R} \neq \mathcal{R}'$ **then**
15: $\quad\quad\quad$ done $\leftarrow$ **true**, $\mathcal{R} \leftarrow \mathcal{R}'$
16: $\quad\quad\quad \pi \leftarrow$ initialize_policy()
17: $\quad$ **if** done **then**
18: $\quad\quad o' \leftarrow$ env_get_initial_state(), $u' \leftarrow u_0$, $\sigma' \leftarrow L(\emptyset, \emptyset, o)$
19: $\quad o \leftarrow o'$, $u \leftarrow u'$, $\sigma \leftarrow \sigma'$
20: **return** π

Algorithm 3 shows a tabu search approach to learn a reward machine. It has the same inputs as local search, but it also receives the size of the tabu list τ_{size}. Our tabu search method is identical to Algorithm 2 except that tabu search initializes the tabu list in line 2, adds the current RM to the tabu list in line 11, and moves to the best solution that is not in the tabu list in lines 12-18.

We note that, technically, these two methods will eventually find an optimal solution. The reason is that both methods restart the search when they get stuck: Local search restarts when it reaches a locally optimal solution and tabu search restarts when it reaches a neighbourhood where all the RMs are in the tabu list. Thus, these methods either find an optimal solution during the search or restart the search.[3] Since every RM can be sampled with equal probability when restarting, both methods will eventually sample an optimal solution (or find one). That said, the space of possible RMs is so vast that we cannot expect our implementations of local search and tabu search to necessarily find optimal solutions in practice.

6. Simultaneously learning a reward machine and a policy

We now describe our overall approach to simultaneously finding an RM and exploiting that RM to learn a policy. Algorithm 4 shows the complete pseudo-code. Our approach starts by collecting a training set of traces $\mathcal{T}$ generated by following a random policy during t_{w} "warmup" steps (line 2). This set of traces is used to find an initial RM $\mathcal{R}$ using one of our discrete optimization models (line 3). The algorithm then sets the RM state to u_0, sets the current high-level observation σ to $L(\emptyset, \emptyset, o)$, and initializes the policy π (lines 4-5). The standard RL loop is then followed (lines 6-19): an action a is selected according to $\pi(a|o, u)$ and the agent receives the next observation o' and the immediate reward r. The RM state is then updated to $u' = \delta_u(u, L(o, a, o'))$ and the last experience (o, u, a, r, o', u') is used to update π. Finally, the environment and RM are reset if a terminal state is reached (lines 17-18).

[3] In the case of tabu search, we are assuming that the tabu list is large enough.

If on any step, there is evidence that the current RM might not be perfect, our approach will attempt to find a new one (lines 11-16). Recall that the RM $\mathcal{R}$ was selected using the cardinality of its prediction sets N, where $N_{u,\sigma}$ is the set of high-level observations seen from the RM state u immediately after observing σ in the training data. If the current high-level observation σ' is not in $N_{u,\sigma}$, then adding the current trace to $\mathcal{T}$ will increase the size of $N_{u,\sigma}$ for $\mathcal{R}$ and, in consequence, $\mathcal{R}$ may no longer be the best RM. Therefore, if $\sigma' \notin N_{u,\sigma}$, we add the current trace to $\mathcal{T}$ and learn a new RM. Our method only uses the new RM if its cost is lower than $\mathcal{R}$'s and, if the RM is updated, a new policy is learned from scratch (lines 14-16).

Given the current RM, we can use any RL algorithm to learn a policy $\pi(a|o,u)$, by treating the combination of o and u as the current state. If the RM is perfect, then the optimal policy $\pi^*(a|o,u)$ will also be optimal for the original POMDP (as stated in Theorem 5). In this case, we can ignore the reward δ_r that comes from the RM and only consider the reward received directly from the environment. However, to further exploit the problem structure exposed by the RM (such as with QRM), we need to set δ_r. We do so using the empirical average, as described in Section 4.2.

Let us now explain how we incorporate QRM into this process. As explained in Section 3, standard QRM under partial observability can introduce a bias because an experience $e = (o, a, o')$ might be more or less likely depending on the RM state that the agent was in when the experience was collected. We partially address this issue by updating Q_u using (o, a, o') iff $L(o, a, o') \in N_{u,\sigma}$, where σ was the current high-level observation that generated the experience (o, a, o'). Hence, we do not transfer experiences from u_i to u_j if the current RM does not believe that (o, a, o') is possible in u_j. For example, consider the cookie domain and the perfect RM from Fig. 2c. If some experience consists of entering to the green room and seeing a cookie, then this experience will not be used by states u_0 and u_3 as it is impossible to observe a cookie at the green room from those states. While adding this rule works in many cases, it does not fully address the problem. We further discuss this issue in Section 9.

7. Experimental evaluation

In this section, we provide an empirical evaluation of our method in three partially observable environments. Our evaluation consists of two parts. First, we compare the effectiveness of our mixed integer linear programming model (MILP), our constraint programming model (CP), our local search with restart algorithm (LS), and our tabu search method (TS) to solve (LRM). We then show how the combination of learning an RM and a policy using double DQN (LRM+DDQN) and deep QRM (LRM+DQRM) compares to different baselines. As a brief summary, our results show the following:

1. MILP and CP find optimal solutions for small instances of (LRM).
2. LS and TS find better solutions than MILP and CP for large instances of (LRM).
3. LS consistently finds better solutions than TS.
4. Our LRM-based methods can outperform A3C, ACER, PPO, and DDQN.
5. LRM+DQRM learns faster than LRM+DDQN, but it is less stable.

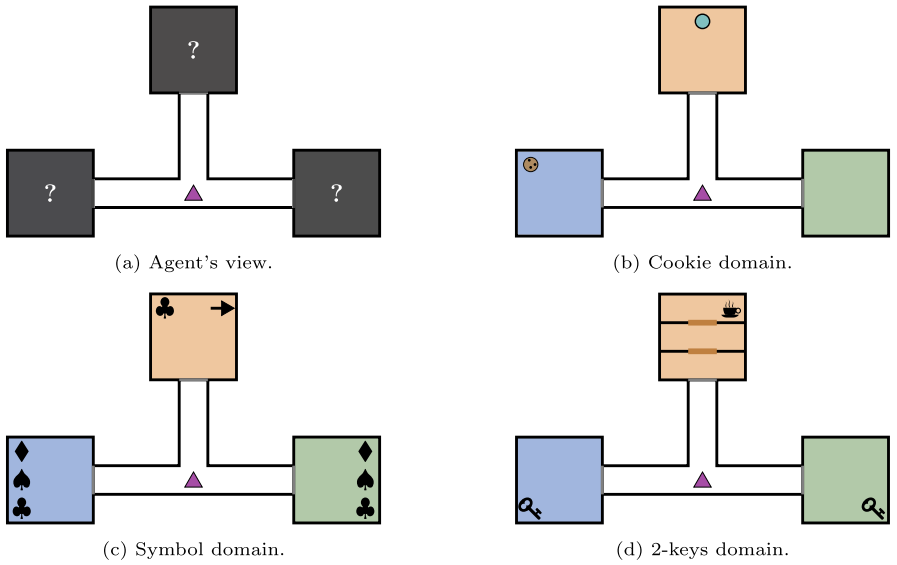
7.1. Domains

We tested our approach on three partially observable domains, shown in Fig. 5. These environments consist of three rooms connected by a hallway. The agent can move in the four cardinal directions but its actions fail with a 5% probability. The length of the hallway is 7, meaning that the agent is required to execute at least 8 actions to go from one room to another. In addition, the agent can only see what it is in the room that it currently occupies, as shown in Fig. 5a. What makes these tasks difficult is the hallway. The hallway forces the agent to observe sequences of identical observations multiple times to solve a task. However, depending on previous observations, the optimal actions and expected returns will be completely different when the agent is in the hallway.

The first environment is the *cookie domain* (Fig. 5b) described in Section 3. Each episode is $5,000$ steps long, during which the agent should attempt to get as many cookies as possible. To do so, it has to press the button in the orange room and then look for the cookie that is delivered to the blue or green room. The agent receives a reward of $+1$ for eating a cookie. All other steps in the environment provide no reward.

The second environment is the *symbol domain* (Fig. 5c). This domain has three symbols ♣, ♠, and ♦ in the blue and green rooms. At the beginning of an episode, one symbol from {♣, ♠, ♦} and possibly a right or left arrow are randomly placed at the orange room. Intuitively, that symbol and arrow will tell the agent where to go, for example, ♣ and → tell the agent to go to ♣ in the east room. If there is no arrow, the agent can go to the target symbol in either room. An episode ends when the agent reaches any symbol in the blue or green room, at which point it receives a reward of $+1$ if it reached the correct symbol and -1 otherwise. All other steps in the environment provide no reward.

The third environment is the *2-keys domain* (Fig. 5d). The agent receives a reward of $+1$ when it reaches the coffee in the orange room. To do so, it must open the two doors, shown in brown. Each door requires a key to open it, and the agent loses the key after opening a door. The agent can only carry one key at a time. At the beginning of each episode, two keys are randomly located in either the green room, the blue room, or split between them. To solve this problem, the agent must keep track of the locations of the keys. The episode ends when the agent reaches the coffee.



(a) Agent's view. (b) Cookie domain.

(c) Symbol domain. (d) 2-keys domain.

Fig. 5. Partially observable domains where the agent can only see what is in the current room.

7.2. Comparisons between the discrete optimization models

We first compare the performance of our four models for solving different instances of (LRM). The objective of this experiment is to compare how well each model scales as we increase the size of the training data and the size of the reward machine. To that end, we generated 20 training sets per domain, where each training set consists of 10^3, 10^4, 10^5, or 10^6 experiences collected by following a uniformly random policy. We sampled five training sets per each possible size and learned reward machines with at most 5 or 10 states. This gave a total of 120 problems instances of (LRM).

Each approach was run with a 10-minute time limit using 62 cores on a Threadripper 2990WX processor with 124GB of RAM. We used Gurobi 9.1 [22] to solve the MILP model and IBM ILOG CP Optimizer 12.8 [27] for the CP model. These are sophisticated state-of-the-art solvers. In contrast, we used a simple Python implementation of local search and tabu search in our experiments. We set $\tau_{\text{size}} = 100$ for tabu search. We note that local search and tabu search are stochastic approaches that, in contrast to MILP and CP, might find a different solution on each run. For that reason, we ran local search and tabu search 5 times per problem instance and report the average cost across those runs.

Table 1 shows the final results considering the best solution found by each model within the time limit. Each row shows the aggregated results over five problem instances that share the same domain (i.e., cookie, symbol, or 2-keys), maximum number of RM states (i.e., $u_{\max} \in \{5, 10\}$), and size of the training set (i.e., $|\mathcal{T}| \in \{10^3, 10^4, 10^5, 10^6\}$). Each row also shows the average size of the training set $|\mathcal{T}_c|$ after the traces are compressed (as described in Section 5). The table reports the average objective function of each model, where lower is better, and the number of instances where each model found the best solution among all others.

For training sets with less than $10,000$ experiences, our CP model tends to find the best solutions. However, for larger instance, local search methods dominate. Note that continuously restarting local search is a better strategy for learning RMs than using a tabu list in these domains. Still, the performance of TS is not too far from LS and, hence, we test both approaches for learning RMs in our next experiments.

7.3. Reinforcement learning experiments

We tested two versions of our *learned reward machine (LRM)* method: LRM+DDQN and LRM+DQRM. Both learn RMs from experience in the same way and solve the (LRM) problem using one of local search with restarts or tabu search. To learn the RM, we used $u_{\max} = 10$, $t_{\max} = 100$, $\tau_{\text{size}} = 100$, and $t_{\text{w}} = 200,000$. The only difference between LRM+DDQN and LRM+DQRM is how they learn the policy $\pi(a|o,u)$ given a learned RM. LRM+DDQN uses the standard DDQN [59] approach to learn $\pi(a|o,u)$. In contrast, LRM+DQRM uses the modified version of QRM described in Section 6. That is, LRM+DQRM exploits the RM structure in order to learn a good policy faster. It does so by reusing the experience collected at the RM state $u \in U$ to also update the policy for a different RM state $u' \in U$, where $u \neq u'$. To learn the policy, we used an epsilon greedy policy with $\epsilon = 0.1$ and a discount factor $\gamma = 0.9$.

We compared against 4 baselines: DDQN [59], A3C [40], ACER [60], and PPO [50]. DDQN uses the concatenation of the last 10 observations as input which gives DDQN a limited memory to better handle the domains. A3C, ACER, and PPO use an LSTM to summarize the history. Note that the output of the labelling function (i.e., the output of the event detectors) was also given to the baselines, as described below.

Table 1
Comparing different models for solving (LRM) in problem instances with training sets varying from 10^3 to 10^6 experiences and $u_{\max} \in \{5, 10\}$. Each row includes five problem instances.

Configuration			Avg. objective				No. best			
Dataset	$\lvert\mathcal{T}\rvert$	$\lvert\mathcal{T}_c\rvert$	MILP	CP	LS	TS	MILP	CP	LS	TS
	10^3	59	**12.6**	**12.6**	14.2	13.8	**5**	**5**	1	1
Cookie	10^4	487	237.3	**226.7**	229.1	230.2	1	**5**	2	0
($u_{\max} = 5$)	10^5	4943	3097.0	2700.4	**2699.7**	2719.7	0	**3**	2	0
	10^6	48663	31075.1	28226.1	**26462.6**	26833.3	0	0	**5**	0
	10^3	59	**6.5**	6.8	9.6	10.5	**5**	4	0	0
Cookie	10^4	487	233.3	204.6	206.0	**204.5**	0	1	0	**4**
($u_{\max} = 10$)	10^5	4943	3197.0	2713.7	**2658.9**	2696.8	0	0	**5**	0
	10^6	48663	30709.5	28366.8	**26461.7**	27092.0	0	0	**5**	0
	10^3	41	**21.0**	**21.0**	**21.0**	**21.0**	**5**	**5**	**5**	**5**
Symbol	10^4	268	**218.8**	**218.8**	**218.8**	220.0	**5**	**5**	**5**	3
($u_{\max} = 5$)	10^5	2597	3423.7	2897.9	**2896.7**	2902.3	0	3	**5**	2
	10^6	25875	36705.4	29689.9	**29687.7**	29688.8	0	4	**5**	3
	10^3	41	**16.2**	**16.2**	16.5	16.4	**5**	**5**	2	2
Symbol	10^4	268	185.8	**181.2**	181.5	185.0	1	**5**	3	0
($u_{\max} = 10$)	10^5	2597	3416.1	2620.7	**2583.5**	2620.4	0	0	**5**	0
	10^6	25875	36216.6	27992.9	**27050.4**	**27050.4**	0	0	**5**	**5**
	10^3	42	**6.9**	**6.9**	7.6	7.5	**5**	**5**	2	2
2-Keys	10^4	378	196.5	**176.7**	176.9	180.7	1	**5**	3	1
($u_{\max} = 5$)	10^5	3690	3713.2	2364.7	**2349.6**	2391.4	0	0	**5**	0
	10^6	37923	38875.3	29379.9	**24397.0**	24762.1	0	0	**5**	0
	10^3	42	**3.5**	**3.5**	5.4	5.1	**5**	**5**	0	0
2-Keys	10^4	378	184.4	151.6	**145.4**	157.2	0	0	**5**	0
($u_{\max} = 10$)	10^5	3690	3746.1	2363.8	**2210.6**	2237.6	0	0	**5**	0
	10^6	37923	38087.0	29065.0	**23352.9**	23558.8	0	0	**5**	0
Average/Total			9732.7	7900.3	**7251.8**	7325.2	38	60	**85**	28

7.3.1. Hyperparameters and features

For LRM+DDQN and LRM+DQRM, the neural network used has 5 fully connected layers with 64 neurons per layer. On every step, we trained the network using 32 sampled experiences from a replay buffer of size 100,000 and a learning rate of $5 \cdot 10^{-5}$. The target networks were updated every 100 steps.

The DDQN baseline uses the same parameters and network architecture as our LRM methods, but its input is the concatenation of the last 10 observations, as commonly done by Atari playing agents [41]. This gives DDQN a limited memory to better handle partially observable domains. We note that since the optimal path from any one room to another is less than 10 steps, giving the agent the last 10 observations means that the agent has enough information to perfectly summarize its history if it can figure out how to do so. The rest of the baselines, namely A3C, ACER, and PPO, use an LSTM to summarize the history.

For A3C, ACER, and PPO, the neural network used has 2 fully connected layers with 512 neurons per layer, all with *tanh* activation functions, and one last LSTM layer with 256 neurons. In addition, all the layers were normalized [5]. These methods collected experiences by running 32 agents in parallel for 128 timesteps. The collected experience was then used to train their network using the Adam optimizer [34] with a learning rate of 10^{-4} and a discount factor of 0.99. To encourage exploration, we added an entropy regularization term with a weight of 0.1. Other hyperparameters are baseline dependent. For PPO, we set the clip rate to 0.1 and the advantage estimation discounting factor to 1. We also trained its network for 4 epochs per update using 2 minibatches. For ACER, we used an experience replay buffer of size $50,000$, sampled 4 batches of data from the replay buffer per batch sampled from the environment, and the maximum KL-divergence between the old and new policy was set to 1. The rest of the hyperparameters were fixed to their default values from the OpenAI baseline implementations [11]. We provide further details on how we tuned the hyperparameters for the baselines in Section 7.3.4.

While interacting with the environment, the agents were given a "top-down" view of the world represented as a set of binary matrices. One matrix had a 1 in the current location of the agent, one had a 1 in only those locations that are currently observable, and the remaining matrices each corresponded to an object in the environment and had a 1 at only those locations that were both currently observable and contained that object (i.e., locations in other rooms are "blacked out"). All agents (both the LRM methods and baselines) also had access to binary features indicating the current status of the events detected by the labelling function.

7.3.2. Hyperparameter tuning

We note that we did not tune the neural network hyperparameters used by the LRM-based approaches. Instead, we simply used the same hyperparameters as in the original RM paper [55].

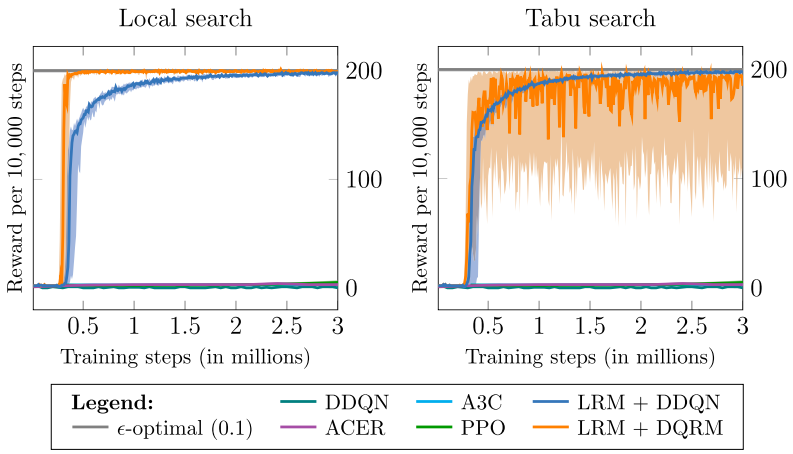


Fig. 6. Results on the cookie domain. LRM is solved using local search or tabu search.

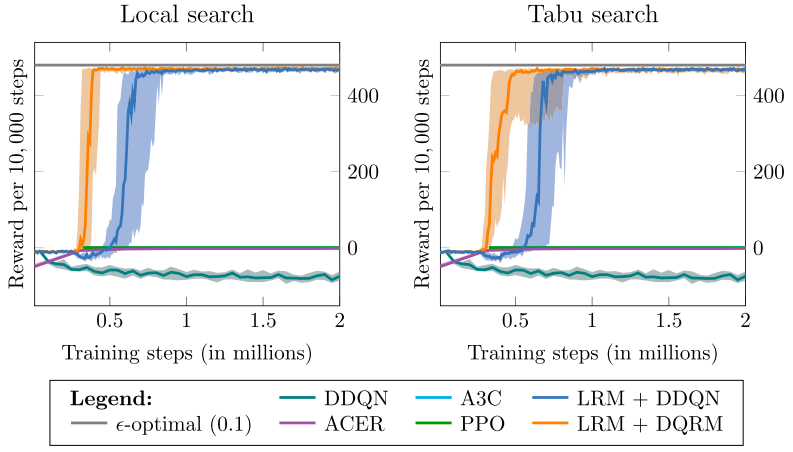


Fig. 7. Results on the symbol domain. LRM is solved using local search or tabu search.

In contrast, we performed an extensive hyperparameter tuning procedure for A3C, ACER, and PPO, using the same methodology that was used in their original publications. That is, we ran each method many times while sampling different values for their hyperparameters. Unfortunately, no run was able to solve our domains by following such an approach, even within 30 million training steps. We then changed the optimizer from RMSprop to Adam, added layer normalization, increased the number of agents, increased the weight of the entropy regularization term, and used a surprisingly large batch size of length 2,048 per update.

Despite this additional tuning, the methods were still unable to find effective policies in 5 million training steps. However, given enough time, PPO was able to find strong policies for two of the three domains. These results are shown in Section 7.3.4. First, in Section 7.3.3, we discuss the results with a shorter time horizon, which allows for a better comparison between local search and tabu search.

7.3.3. Results

Figs. 6, 7, and 8 show the total cumulative rewards that each approach gets every 10,000 training steps and compares it to the ϵ-optimal policy. For all the algorithms, the figures show the median performance over 30 runs per domain, and percentile 25 to 75 in the shadowed area. Each figure shows two settings. In the left, it shows the performance when LRM is solved using local search with restarts. In the right, it shows the case where LRM is solved using tabu search. Note that this only affects the LRM methods. The baselines' performance is identical in the left and right figures.

The figures also show the performance of an ϵ-optimal policy, that is, the performance of an optimal policy that explores the environment using ϵ-greedy with $\epsilon = 0.1$. Note that the performance of the ϵ-optimal policy is an upper bound for the performance of any RL method that explores the environment using ϵ-greedy exploration. This includes the LRM methods and DDQN baseline in this experiment. Moreover, reaching the ϵ-optimal performance suggests that the agent converged to an optimal policy.

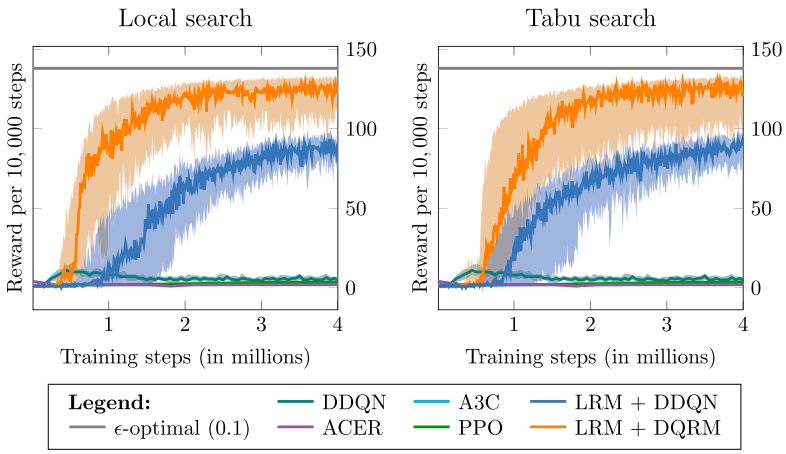


Fig. 8. Results on the 2-keys domain. LRM is solved using local search or tabu search.

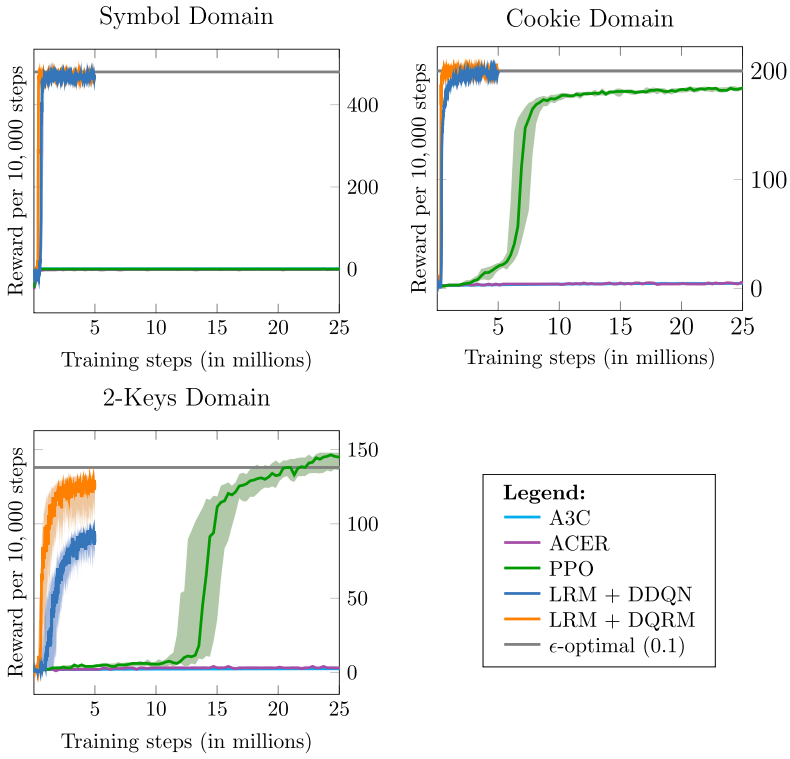


Fig. 9. More results on the three domains. LRM is solved using local search; the results for it are repeated from previous experiments. The other methods were run for more training steps.

As the results show, LRM-based methods outperform all the baselines in these domains, reaching the performance of the ϵ-optimal policy in the cookie and symbol domains (see Figs. 6 and 7). We also note that LRM+DQRM learns faster than LRM+DDQN. In particular, LRM+DQRM converged to considerably better policies in the 2-keys domain (Fig. 8). However, LRM+DQRM is more unstable than LRM+DDQN when solving (LRM) via tabu search. We believe this behaviour is due to two factors. First, tabu search is likely finding worse solutions than local search, as suggested by Table 1. Second, QRM exploits the structure of the learned RM. Thus, it is reasonable to expect that converging to a suboptimal RM would hurt the performance of DQRM more than the performance of DDQN.

Note that we ran all the experiments for five million training steps, but cut off the graphs (in Figs. 6, 7, and 8) earlier when there were no longer interesting changes, to focus on the learning process. We describe results for longer training times in the next section and its Fig. 9.

7.3.4. Evaluating the LSTM-based methods

In this section, we provide more details about the comparison made between the LRM and the LSTM-based methods. Recall that, in contrast to the LRM-based approaches, we performed an extensive hyperparameter tuning procedure for A3C, ACER, and PPO. Despite this additional tuning, the methods were still unable to find effective policies in 5 million training steps. However, given enough time, we did find that PPO was able to find strong policies for two of the three domains.

Fig. 9 shows the performance of the LSTM-based approaches when run for up to 25 million steps (we kept the LRM-based experiments to only 5 million steps). Notice that in the 2-keys domain, PPO outperforms the ϵ-optimal policy – which is possible since PPO does not explore the environment using ϵ-greedy. In addition, PPO converged to a strong suboptimal policy in the cookie domain, and PPO was ineffective in the symbol domain. A3C and ACER still made no progress even after 25 million training steps.

We believe that PPO with an LSTM struggles on the symbol domain because the likelihood of guessing the right symbol is only $\frac{2}{9}$. This is because the agent has a $\frac{1}{3}$ chance of picking the right symbol type, and a $\frac{2}{3}$ chance of picking an appropriate room (either by happening to pick the room that was specified, or because no specific room was specified). Thus, the expected immediate reward for going to a random symbol is $1 \cdot \frac{2}{9} + (-1) \cdot \frac{7}{9} = -\frac{5}{9}$. This penalty encourages PPO to stay away from the symbols since all other steps have a reward of 0. This can make it very difficult for the LSTM-based agents to learn the connection between the given instructions about which symbol to go to, and the reward provided by each of the symbols.

Unlike the LSTM-based approaches, LRM has no problem solving the symbol domain because LRM divides the overall task into two parts. The first part focuses on learning the high-level relationships between the events that occur in the environment. This includes learning the relationship between the instruction and the reward provided by each symbol. Once the agent understands that connection, LRM focuses on learning a policy that tries to collect as much reward as possible from the environment.

7.3.5. Experiments using long hallways

As discussed in Section 7.1, what makes our tasks difficult is the hallway. Hallway-like mazes are good sandboxes for comparing the memory capabilities of RL agents because RL agents tend to struggle when facing tasks that require remembering key information for many time steps in order to solve a given task [43,44]. The hallway further complicates the problem by showing the agent the same sequence of observations as it moves through the hallway. As such, the longer the hallway, the harder the task generally is for the agents.

In this section, we examine how the performance of the RL agents changes as we increase the length of the hallway from 7 (the length in our previous experiments) to 21. Note that if the length of the hallway is x, then it means the agent is required to execute at least $x + 1$ actions to go from one room to another.

Since these experiments take longer to run, we only compare the performance of the best method from the previous experiment: our LRM methods using local search, and PPO using an LSTM. As in the previous experiment, we did not further tune the hyperparameters for the LRM methods, but we did perform extensive hyperparameter tuning for PPO.

The results on the larger domains are shown in Fig. 10. As expected, the performance of all the methods decreased but LRM generally outperforms PPO. In the cookie domain, we see a larger benefit from using QRM than before. However, the use of QRM leads to more unstable learning in the symbol domain, where LRM+DDQN found better policies after 10 million training steps. We believe that this behaviour is due to two factors. On the one hand, having a longer hallway makes QRM more effective since the agent can reuse its experience to learn how to navigate the hallway independently of the current RM state. On the other hand, having a longer hallway makes it harder for the agent to find traces that represents sufficiently well the dynamics of the environment. As a result, the quality of the RMs learned by LRM decreased. When that happens, the fact that QRM more aggressively exploits a given RM — which is of lower quality in this case — makes the learning process more unreliable. Finally, we note that none of the methods were able to find an effective policy for the 2-keys domain within 10 million training steps. While QRM may be making some progress after the 10 million time step point, it is still well below the performance of a ϵ-greedy optimal value on this problem.

We also ran PPO for a longer set of time steps — 60 million in this case — to see if it eventually could solve these large hallway problems, as it did in the previous section. However, in all three domains, PPO was unable to make any progress given this additional time, regardless of the hyperparameter settings used. This is in line with previous work which have also reported similar behaviour for LSTM-based agents as the length of hallways increased [e.g., 43,44].

7.3.6. Experiments using different warmup steps for learning the reward machine

LRM methods rely on learning RMs that model the (partially-observable) dynamics of the environment. But the quality of the learned RM depends on the quality of the training data. If the training set does not represent the environment's dynamics, then the learned RM will provide limited utility to an RL agent.

To ensure learning high-quality RMs, LRM methods rely on two strategies. First, the initial set of training traces is collected by following a uniform random policy for t_w warmup steps. If t_w is large enough, the set of training traces will contain all the required information to learn a high-quality RM. Second, LRM methods constantly monitor the correctness of the current RM. When the RM makes a wrong prediction, they add that counterexample to the set of training traces and relearn the RM. In this section, we test the effectiveness of this methodology.

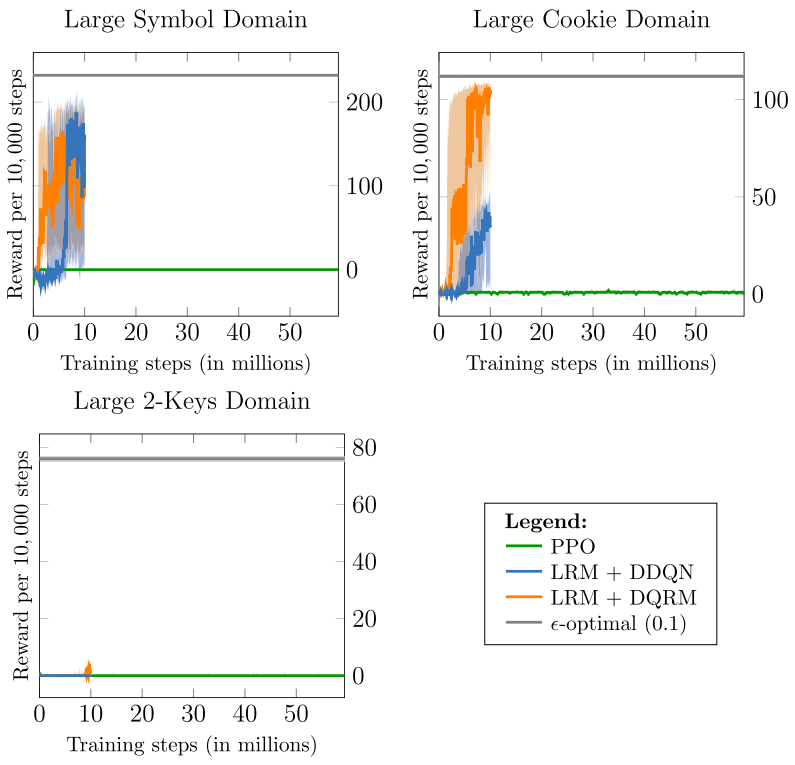


Fig. 10. Results on the large version of the domains. LRM is solved using local search. In this experiment, LRM methods were run for 10 million steps, and PPO was run for 60 million steps.

Fig. 11 shows the performance of LRM+DDQN using different numbers of warmup steps. In the experiment, LRM was solved using our local search method with random restarts. Each line represents the median performance across 30 seeds, and the shadowed area shows the 25th and 75th percentiles.

Overall, the results show that LRM is able to find strong policies in all the test domains, regardless of the number of warmup steps. However, using a small number of warmup steps results in a higher variance across the runs. This variance is due to insufficient training data at the moment of learning the first RM. As the agent explores the environment, it discovers that its RM is not perfect and relearns the RM. Depending on multiple factors, each run might relearn the RM several times before solving the problem – causing the variance observed in the plot. On the other hand, if we use a large number of warmup steps then the first RM is often close to perfect, and, as such, there is no need to relearn it.

To further illustrate this point, Table 2 shows the average number of times that the RM is relearned as we vary the number of warmup steps. A value of 0 means that the first RM found was perfectly accurate, and our system never found a second one. Notice that if we set t_w to 1 million, the RM is rarely relearned in our domains. In contrast, if we set t_w to ten thousand, the RM is relearned from 20 to 75 times per run, depending on the domain.

8. Comparison with alternative methods for learning reward machines

Other researchers have also explored the topic of learning RMs (and other forms of finite state machines) in order to improve the performance of RL agents [e.g., 62,63,16,48,17,38,42,23,9,12]. All of these methods learn finite state machines from traces that are collected by an agent, and then use some RL algorithm to learn a policy for selecting actions given the current environment and automata state. However, these methods differ in how they define the optimal automata given a set of traces (i.e., how they define their objective function or the target automata of their optimization) and how they search for that automata. Each can be seen as being aimed at a certain class of problems. Below, we will compare and contrast these methods, and identify why of all of them, LRM is best suited for handling partial observability.

8.1. Modelling the reward function

The most common learning objective is to learn an RM that focuses solely on making accurate predictions of the reward signal [62,63,16,48,17,38,42,9,12]. For instance, JIRP [62] is a SAT-based program that learns the smallest RM that perfectly predicts the reward provided by the environment. These methods are aimed at solving *Non-Markovian Reward Decision Processes (NMRDPs)* [6]. An NMRDP is a tuple $\mathcal{N} = \langle S, A, r, p, \gamma, \mu \rangle$, where S, A, p, γ, and μ are defined as in an MDP.

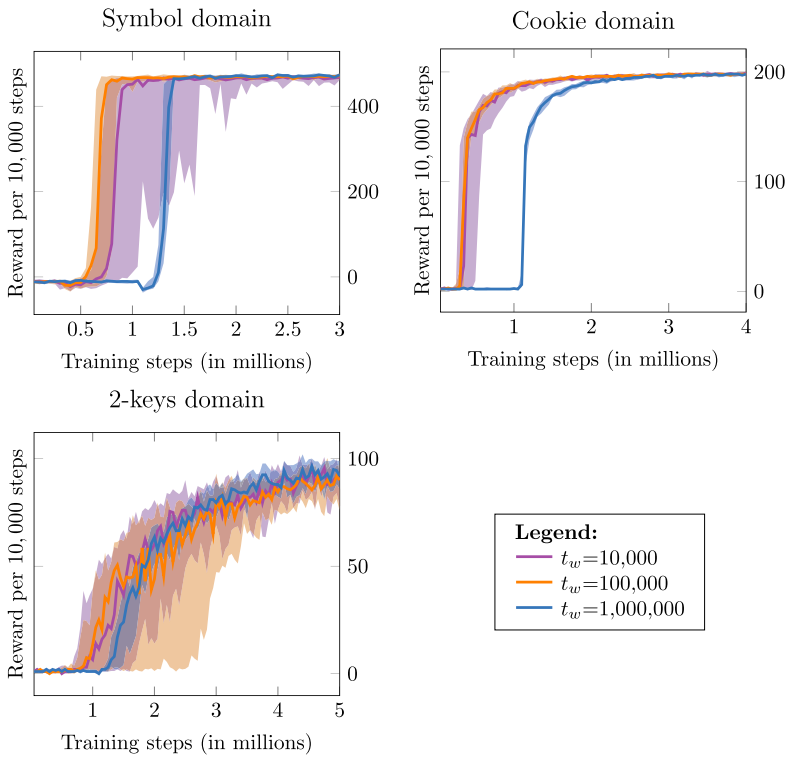


Fig. 11. Results on the three domains. Each line shows the performance of LRM+DDQN as we vary the number of warmup steps for learning the initial RM.

Table 2
The average number of times that LRM+DDQN re-learned the RM within a run as we vary the number of warmup steps. These results are the average across 30 runs.

LRM+DDQN	Cookie	Symbol	2-Keys
$t_w = 10,000$	20.97	74.73	45.90
$t_w = 100,000$	6.07	24.77	28.97
$t_w = 1,000,000$	0.13	0.00	1.60

But the reward function $r : (S \times A)^* \to \mathbb{R}$ is a function of the entire history of the agent's interaction in the environment. That is, the transition probabilities are Markovian and the state of the environment $s \in S$ is known by the agent, but the immediate reward $r_{t+1} = r(s_0, a_0, \cdots, s_t, a_t, s_{t+1})$ is non-Markovian. As a result, the optimal policy for an NMRDP might need to consider the whole history of states and actions $\pi^*(a_t|s_0, a_0, \cdots, s_t)$ to decide what action to perform next.

To simplify the task of learning a history-based policy $\pi^*(a_t|s_0, a_0, \cdots, s_t)$, methods like JIRP learn automata that can make Markovian predictions of the non-Markovian reward function. Generally, these methods focus on learning the smallest automata which has this property. Then, these methods construct an MDP given by taking the cross-product of the environment states and the automata state. This MDP is equivalent to the original NMRDP [63], but now allows standard MDP-based RL methods to be used to find a policy $\pi^*(a_t|s_t, u_t)$, where u_t is the current state of the learned RM, which can be used for the original NMRDP.

However, by focusing on NMRDPs, JIRP-like methods will generally be ineffective for POMDPs. To see why, recall that while the state transitions in a POMDP are Markovian, the agent only has access to partial information about that state in the form of an observation. As such, from the perspective of the agent, both the reward function and the observations might depend on the complete history of the agent's interaction with the environment. That is, the observation transition function is a probability distribution is given by $P(o_{t+1}|o_0, a_0, ..., o_t, a_t)$.

In order to apply any of the existing NMRDP-focused approaches to a POMDP, these methods must treat the observations as states. In their current form, doing so means assuming that $P(o_{t+1}|o_0, a_0, ..., o_t, a_t) = P(o_{t+1}|o_t, a_t)$. To see the issues with doing so, consider the cookie domain. In this domain, the reward function is actually Markovian with respect to the observations: the agent gets a reward whenever they are at the same location as a cookie (since they eat it), and otherwise they get 0. Since most of the NMRDP-focused approaches aim to find the smallest RM consistent with the reward function, they will therefore generate the one-state reward machine given in Fig. 2a. However, this reward machine does not account

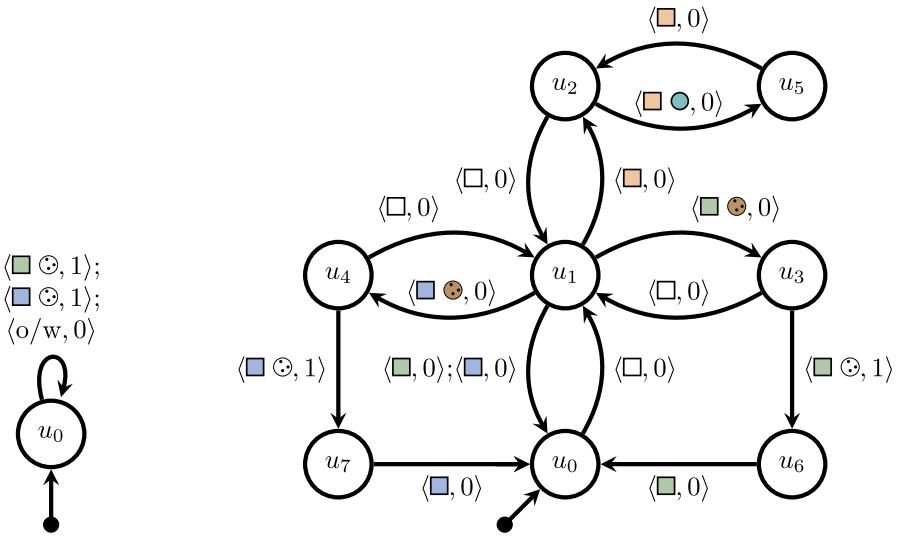


Fig. 12. Reward machines learned by alternative approaches to learn RMs in the cookie domain.

for the fact that the probability that the agent is able to reach an observation will depend on the agent's history. For example, the probability of entering the blue room and seeing a cookie is 0 if the button has never been pushed, but non-zero in some other circumstances in which the button has been pushed. Since the single-state reward machine will not capture that the button needs to be pushed first, the NMRDP-focused approaches will not be useful for generating an effective policy since they do not explicitly take into account that the probability of different sequences of transitions also depends on the agent's history.

We verified that this behaviour was happening empirically by using SRMI [9] – a recently proposed extension of JIRP – to learn an RM for the cookie domain. The result is shown in Fig. 12a. As expected, the smallest RM that perfectly models the reward signal in the cookie domain has only one state. In that state, the RM predicts a reward of one when the agent eats the cookie and zero otherwise. This simple RM is enough to make perfect predictions of the reward signal, but it is not enough to make the cookie domain Markovian with respect to the cross-product between the agent's observation and the RM state. For that reason, JIRP, SRMI, and any other method that learns the smallest RM that models the reward signal is not going to work well when facing general POMDPs.

On the other hand, we do not expect LRM to perform well in NMRDPs because its objective function does not optimize toward making the reward signal Markovian. To correctly use LRM in problems where the reward signal is non-Markovian we would have to penalize incorrect reward predictions in LRM's objective function. But even in that case, we do not expect LRM to outperform JIRP-like methods when solving NMRDPs because LRM is not tailored to solve NMRDPs.

8.2. Methods that learn a high-level model of the environment

In contrast to methods that model the reward signal, DeepSynth [23] aims to solve MDPs with sparse rewards by learning a finite state machine – which could be easily transformed into an RM – that models the environment's dynamics from the perspective of the high-level observations given by the labelling function.

Their learning objective consists of finding the smallest machine that accepts any (high-level) traces that can be generated by the agent while interacting with the environment. And, at the same time, reject any trace that cannot be generated while interacting with the environment. For instance, the following (compressed) trace from the cookie domain should be accepted: $\tau^+ = (\square, \blacksquare, \blacksquare\bullet, \blacksquare, \square, \blacksquare\text{⚉})$, but the trace $\tau^- = (\square, \blacksquare, \square, \blacksquare\text{⚉})$ should be rejected because it is not possible to see a cookie without pressing the button.

To do so, DeepSynth [23] first collects traces from interacting with the environment and then uses an automata learning method called Trace2Model [30] to learn the machine. Trace2Model considers that every collected trace is a positive example and uses other traces as negative examples. Then, it uses a SAT solver to learn a deterministic finite state machine that accepts all the positive traces and rejects all the negative traces. After learning the machine, DeepSynth learns one policy per state of the machine and also provides intrinsic motivation (in the form of extra reward) when a new high-level observation is encountered for the first time in an episode. These extra rewards encourage exploration in environments with sparse rewards.

We note that the learning objectives of DeepSynth's Trace2Model approach and LRM are similar. They both optimize towards finding a machine whose state can be used to predict the next high-level observation. However, there are two key differences.

First, Trace2Model cannot directly handle long traces. The reason is that the space of training traces is huge. If there are P propositions, then the number of possible traces is $O(2^{P \cdot H})$ where H is the maximum length of a trace. If Trace2Model were to consider all those traces as either positive or negative examples when learning the machine, that would be intractable.

To overcome this practical limitation, Trace2Model introduces two hyperparameters. The first hyperparameter w – which is set to 3 in the experiments in the DeepSynth paper – controls the length of the positive traces. All the positive traces are cut in subsequences of w consecutive observations using a sliding window. For instance, the positive trace $\tau^+ = (\square, \square, \square\bigcirc, \square, \square, \square\cookie)$ will result in the following subtraces: $(\square, \square, \square\bigcirc)$, $(\square, \square\bigcirc, \square)$, $(\square\bigcirc, \square, \square)$, and $(\square, \square, \square\cookie)$. Then, the subtraces are used to learn the machine. The full trace is only added as a constraint if the automata learnt using only subtraces does not accept it. The second hyperparameter l – which is set to 2 in the experiments in the DeepSynth paper – controls the length of the negative traces. For instance, in the cookie domain a negative trace of length 2 would be $(\square, \square)$ because the agent cannot go from the green room ($\square$) to the orange room ($\square$) without passing through the hallway ($\square$).

By setting w and l to small values, Trace2Model is a practical approach for learning automata. However, using small values for w and l limit the patterns that Trace2Model can learn. For instance, to learn that the cookie is not going to disappear if we leave the room, we need to consider the following trace as a negative example: $(\square\cookie, \square, \square)$, which can only be considered if we set $l \geq 3$. And learning more complicated patterns (such as the relationship between pressing the button and observing a cookie) will require a considerably larger value of l.

To verify this, we used Trace2Model [30] to learn an RM for the cookie domain using the standard parameters of $w = 3$ and $l = 2$. The result is shown in Fig. 12b. Notice that the resulting RM correctly encodes many of the Markovian relationships in the environment. For instance, it learned that from the hallway (state u_1) it is possible to go to any of the other rooms, from the orange room (state u_2) it is possible to press the button, etc. However, it did not learn the relationship between pressing the button (state u_5) and observing a cookie (states u_4 and u_5) because these require larger values for the method's hyperparameters. As a result, this RM is not sufficient to solve the cookie domain.

In principle, using a larger value of l should allow Trace2Model to learn more useful RMs for the cookie domain. But we ran Trace2Model with $l = 3$ for more than a day and it was unable to find an RM that could model the training traces and reject all the negative traces of length 3. This result is consistent with the fact that the problem of finding a deterministic machine of a minimum number of states compatible with a given finite set of positive and negative examples is NP-hard [21,3]. Since the number of training traces increases exponentially with l, learning an RM using Trace2Model with $l = 3$ can take a prohibitively long time.

There is a second difference between LRM and Trace2Model that makes LRM more suitable to solve partially observable problems. To solve a POMDP, the state of the RM should uncover all the necessary information to make the problem Markovian. To that end, LRM's learning objective is to find an RM that allows the agent to predict the next high-level observation $L(e_{t+1})$ given the current high-level observation $L(e_t)$ and RM state u_t. In contrast, Trace2Model learns a machine that can predict $L(e_{t+1})$ only from the current state u_t.

For instance, in the cookie domain, LRM prefers RMs that remember when the agent pressed the button ($\square\bigcirc$), because then the RM can better predict what is possible when the agent is in the hallway ($\square$). However, since observing the hallway itself does not add any more information that will change the transition probabilities of the problem, it does not need an additional RM state to capture that the hallway was seen after the button was pushed. In contrast, the automaton learned by Trace2Model will have an explicit state capturing that the agent is currently in the hallway (u_1 in Fig. 12b).

Therefore, even if Trace2Model were able to learn a perfect RM for the cookie domain, that machine will be much larger than the minimal perfect RM for the cookie domain. Indeed, the RM from Fig. 12b is far from a perfect RM and it already has 8 states – even though we know that there exists a perfect RM with only 4 for the cookie domain. More generally, a minimal perfect RM will be an optimal solution to LRM (given the conditions detailed in Theorem 6), but not necessarily for Trace2Model. Since the complexity of solving Trace2Model increases with the size of automaton, being unable to learn the minimal perfect RM further constrains the practical applicability of DeepSynth in POMDPs.

8.3. Empirically comparing RM-learning approaches on POMDPs

To verify the above analysis outlining why LRM is better suited for POMDPs than other approaches for automata learning for RL, we empirically compared several methods on our partially observable environments. In particular, we compare with DeepSynth [23] and SRMI [9], the latter of which is a state-of-the-art representative of the methods that learn RMs to solve NMRDPs.

To make the comparison fair, we provided each learning method with the same set of training traces. In our experiments, for each domain we used the five training sets with one million experiences from Section 7.2. To learn an RM, we let SRMI and DeepSynth run until converging to an optimal solution. In the case of LRM, we used a time limit of ten minutes. We then gave the learned RM to DDQN and learned a policy $\pi_\theta(a|o, u)$ that considered the current observation and RM state to decide the next action. We trained DDQN for 3 million training steps using the same hyperparameters from Section 7.3.1.

Table 3 shows the average reward obtained by each method during the last 10,000 training steps (averaged over five runs for each of the five training sets of traces). As expected, LRM is the only approach that is able to learn an RM that allows DDQN to solve these partially observable domains.

Table 3
Comparing different methods for learning RMs under partial observability. We provided each method a fixed set of training traces and let them learn an RM. We then use DDQN to learn a policy $\pi(a|o,u)$ that tries to solve the partially-observable problems using the learned RM. Each cell shows the average reward and standard deviation across 25 experiments.

Method	Cookie	Symbol	2-Keys
SRMI [9]	0.6 ± 0.8	-31.1 ± 13.5	2.0 ± 1.4
DeepSynth [23]	0.4 ± 0.6	-30.4 ± 14.0	2.4 ± 1.5
LRM (ours)	**197.2** ± **2.1**	**460.4** ± **15.7**	**86.6** ± **9.4**

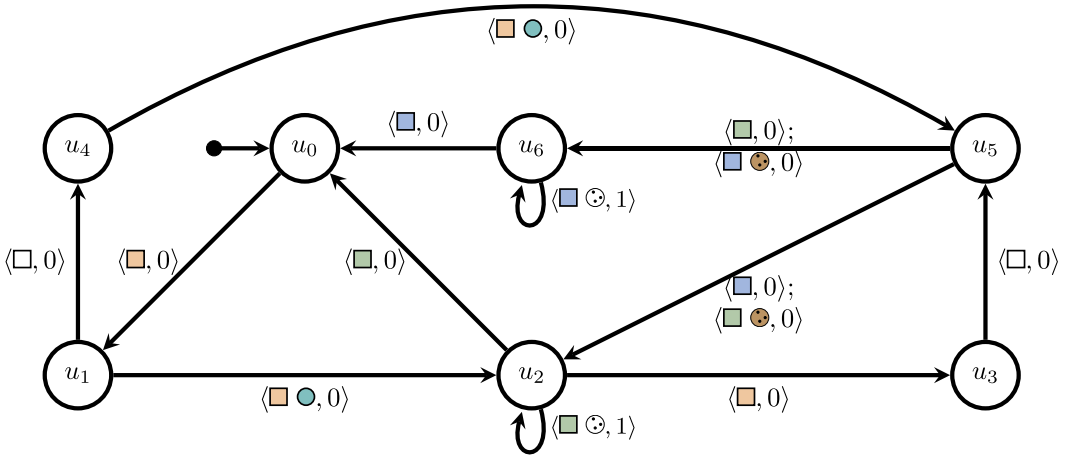


Fig. 13. A randomly picked RM learned by our local search method in the cookie domain. Self-loops were removed for clarity. That is, every state has an implicit self-loop transition with label $\langle o/w, 0 \rangle$.

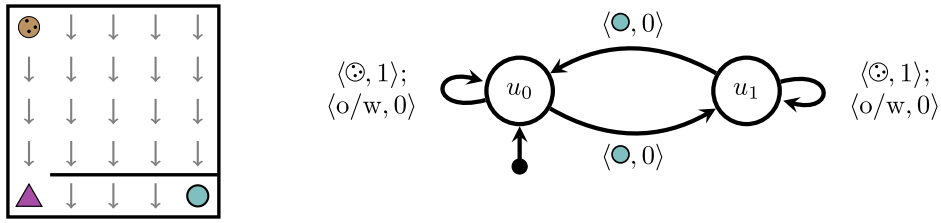
8.4. What is LRM learning?

Our results show that LRM consistently learns RMs that allow RL agents to obtain high returns when solving partially observable problems. Our local search method achieves that goal by stochastically searching for an RM (with up to $U_{\max}$ states) that makes accurate predictions of future observations.

In contrast to SRMI and DeepSynth, our local search method outputs a different RM in every run – even if the set of training traces is identical in each run. Thus there is no one unique RM that LRM learns. We know that all these RMs are capable of keeping track of the important parts of the observation history, though (otherwise, LRM+DDQN would not get high returns). As an example, Fig. 13 shows one RM learned by local search in the cookie domain. This RM was randomly selected from the set of over 50 RMs that were learned by local search in the cookie domain across all our experiments. All the RMs that were learned across all our experiments can be found in our source code repository.

If we examine the RM from Fig. 13 we see why this RM allows RL agents to solve the cookie domain. As explained in Section 4, to make the cookie domain Markovian the RM must keep track of the locations of the cookies using its current state. The Fig. 13 RM begins in u_0 and moves to u_1 as soon as the agent enters the orange room – which is the room that has the button. From u_1 there are two possible paths to u_5 – which is the state where the agent knows that there is a cookie in either the green room or the blue room. If the agent presses the button, the RM moves to state u_2 and then to state u_3 as soon as the agent stops pressing the button. From u_3 the machine can only move to u_5 once the agent leaves the orange room. Alternatively, the agent in u_1 could have gone back to the hallway and reach u_4 and from u_4 reach u_5 when it presses the button. Crucially, regardless of the path taken by the agent to u_5, being in u_5 means that the agent pressed the button and, thus, there is a cookie somewhere. From u_5 the agent moves to u_6 when it knows that the cookie is in the blue room and to u_2 when it knows that the cookie is in the green room. Finally, from either u_2 or u_6 the RM moves back to u_0 once the room where the cookie was located no longer contains a cookie – which means that the agent ate the cookie.

That said, we note that the RM from Fig. 13 is not a perfect RM. For instance, if the agent gets to u_5, observes the cookie in the green room (moves to u_2) and, instead of eating the cookie, goes back to the orange room (moves to u_3) then the agent will forget the location of the cookie. But that bug is unlikely to cause troubles to an RL agent that is using that RM because once the agent knows that the cookie is in the green room the optimal course of action is to go there and eat it.



(a) The gravity domain. (b) A perfect RM for the gravity domain.

Fig. 14. A partially observable environment where the agent cannot see the external force.

9. Discussion

Solving partially observable RL problems is challenging and LRM was able to solve three problems that were conceptually simple but presented a major challenge to A3C, ACER, and PPO with LSTM-based memories. A key idea behind these results was to optimize over a necessary condition for perfect RMs. This objective favours RMs that are able to predict possible and impossible future observations at the abstract level given by the labelling function L. In this section, we discuss the advantages and current limitations of such an approach.

We begin by considering the performance of local search methods in our domains. Given a training set composed of one million transitions, our simple Python implementation of local search takes less than 2.5 minutes to learn an RM across all our environments, when using 62 workers on a Threadripper 2990WX processor and $t_{\max} = 100$. Note that local search's main bottleneck is evaluating the neighbourhood around the current RM solution. As the size of the neighbourhood depends on the size of the set of propositional symbols $\mathcal{P}$, exhaustively evaluating the neighbourhood may sometimes become impractical. To handle such problem, we might import ideas from the *large neighbourhood search* literature [46].

Regarding limitations, learning the RM at the abstract level is efficient but requires ignoring (possibly relevant) low-level information. For instance, Fig. 14a shows an adversarial example for LRM. The agent receives reward for eating the cookie (☹). There is an external force pulling the agent down – i.e., the outcome of the "`move-up`" action is actually a downward movement with high probability. The agent can press a button (●) to turn off (or back on) the external force. Hence, the optimal policy is to press the button and then eat the cookie. Given $\mathcal{P} = \{☹, ●\}$, a perfect RM for this environment is fairly simple (see Fig. 14b) but LRM might not find it, even if the traces are not compressed. The reason is that pressing the button changes the low-level probabilities in the environment but does not change what is possible or impossible at the abstract level. In other words, while the LRM objective optimizes over necessary conditions for finding a perfect RM, those conditions are not sufficient to ensure that an optimal solution will be a perfect RM. In addition, if a perfect RM is found, our heuristic approach to share experiences in QRM would not work as intended because the experiences collected when the force is on (at u_0) would be incorrectly used to update the policy for the case where the force is off (at u_1).

We note that our current experiments do not consider how the reward machine-learning approach of LRM can be combined with other RL methods, especially those like PPOs that use LSTMs. We believe this is an important area of future work that will help clarify further the advantages and disadvantages of using LRM versus using an LSTM-based memory. In particular, it may potentially show that these different forms of memories can be combined to best introduce exploration and handle different types of partial observability.

Other current limitations include that it is unclear how to handle noise over the high-level detectors L and how to transfer learning from previously learned policies when a new RM is learned. Finally, defining a set of proper high-level detectors for a given environment might be a challenge to deploying LRM. Hence, looking for ways to automate that step is an important direction for future work.

10. Related work

State-of-the-art approaches to partially observable RL use Recurrent Neural Networks (RNNs) as memory in combination with policy gradient [e.g., 40,60,50,29] or use external neural-based memories [e.g., 43,33,26]. Other approaches include extensions to Model-Based Bayesian RL that work under partial observability [e.g., 47,13,18] or provide a small binary memory to the agent and a special set of actions to modify it [45]. While our experiments highlight the merits of our approach with respect to RNN-based approaches, we rely on ideas that are largely orthogonal. As such, there is significant potential in mixing these approaches to get the benefit of memory at both the high- and the low-level.

The effectiveness of automata-based memory has long been recognized in the POMDP literature [8], where the objective is to find policies given a complete specification of the environment. The idea is to encode policies using *finite state controllers (FSCs)* which are *finite state machines (FSMs)* where the transitions are defined in terms of low-level observations from the environment and each state in the FSM is associated with one primitive action. When interacting with the environment, the agent always selects the action associated with the current state in the controller. Meuleau et al. [39] adapted this idea to work in the RL setting by exploiting policy gradient to learn policies encoded as FSCs. RMs can be considered as a generalization of FSC as they allow for transitions using conditions over high-level events and associate complete policies

(instead of just one primitive action) to each state. This property allows our approach to easily leverage existing deep RL methods to learn policies from low-level inputs, such as images – which is not achievable by Meuleau et al. [39]. That said, further investigating using ideas for learning FSMs [e.g., 4,64,19,51] in learning RMs is a promising direction for future work.

Our approach to learn RMs is greatly influenced by *predictive state representations (PSRs)* [36]. The idea behind PSRs is to find a set of core tests (i.e., sequences of actions and observations) such that if the agent can predict the probabilities of these occurring, given any history H, then those probabilities can be used to compute the probability of any other test given H. The insight is that state representations that are good for predicting the next observation are good for solving partially observable environments. We adapted this idea to the context of RM learning.

Finally, as noted above, several different approaches to learn RMs were proposed simultaneously, or shortly after, our original publication [e.g., 62,63,16,48,17,38,42,23,9,12]. They all learn reward machines in either MDPs or NMRDPs. In most of these works, the goal is to learn the smallest RM that is consistent with the reward function. To do so, they usually use off-the-shelf automata learning approaches to learn the RM. These include methods that learn reward machines by using a SAT solver [62,42], inductive logic programming [16], and program synthesis [23]. There has also been work on adapting the L^* algorithm [3] to learn RMs given the model of the MDP [48], expert demonstrations [38], or in a pure RL setting [17,63].

Besides proposing approaches to learn reward machines for NMRDPs, these works make additional contributions that may be useful in the context of solving POMDPs. For instance, Furelos-Blanco et al. [16] and Hasanbeig et al. [23] add a reward shaping procedure to encourage exploration. Xu et al. [62] propose a simple mechanism to transfer some of the previously learned Q-value estimates when a new reward machine is learned. Neider et al. [42] show how to incorporate domain knowledge when learning a reward machine. Gaon and Brafman [17] and Xu et al. [63] allow, in some cases, the agent's exploration to be driven towards finding *bugs* in the RM. Finally, Corazza et al. [9] and Dohmen et al. [12] recently proposed stochastic versions of RMs and showed how to learn them. Further study into how to use these ideas in the case of partial observability is left as future work.

11. Concluding remarks

RL is proving to be an effective technique for building sequential decision making systems, particularly in cases where environment dynamics are unknown or difficult to model by hand. Unfortunately, the effectiveness of such techniques is often predicated on the environment being fully observable, and yet the majority of real-world applications occur in partially observable environments. In this paper we proposed a method to perform reinforcement learning in partially observable environments by learning a reward machine – an automata-based structure that serves as structured external memory, augmenting observed state in order to provide a (near) Markovian decomposition of the RL problem. We formulated the RM learning task as a discrete optimization problem that optimized a set of RM properties informed by criteria from the POMDP, FSC, and PSR literature. We experimented with several optimization methods, finding local search methods to be the most effective for learning RMs. We then combined this RM learning with policy learning for solving partially observable RL problems. Experiments showed impressive and significant improvements in sample efficiency relative to LSTM-based techniques. Indeed our approach solved problems that were unsolvable by A3C and ACER, even after 25 million training steps, and with PPO requiring an order of magnitude more training steps than our method in the subset it could solve. Experiments similarly showcased the strength of our method, relative to PPO, when what needed to be remembered was in the distant past.

We believe this work represents an important building block for creating RL agents that can solve cognitively challenging tasks in partially observable environments. RM learning provides the agent with memory, but more importantly the combination of RM learning and policy learning provides it with discrete reasoning capabilities that operate at a higher level of abstraction, while leveraging deep RL's ability to learn policies from low-level inputs. This work leaves open many interesting questions relating to abstraction, observability, and properties of the language over which RMs are constructed. We believe that addressing these questions will push the boundary of partially observable RL problems that can be solved.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We have shared the link to our code in the paper.

Acknowledgements

We gratefully acknowledge funding from the Natural Sciences and Engineering Research Council of Canada (NSERC), the Canada CIFAR AI Chairs Program, and Microsoft Research. We also acknowledge funding from the National Center for Artificial Intelligence CENIA FB210017 (Basal ANID), FONDECYT grant 11230762, and FONDECYT grant 11230797. Resources used in preparing this research were provided, in part, by the Province of Ontario, the Government of Canada through CIFAR,

and companies sponsoring the Vector Institute for Artificial Intelligence (vectorinstitute.ai/partners/). Finally, we thank the Schwartz Reisman Institute for Technology and Society for providing a rich multi-disciplinary research environment.

References

[1] E. Aarts, E.H. Aarts, J.K. Lenstra, Local Search in Combinatorial Optimization, Princeton University Press, 2003.
[2] M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, et al., Learning dexterous in-hand manipulation, CoRR, arXiv:1808.00177, 2018.
[3] D. Angluin, Learning regular sets from queries and counterexamples, Inf. Comput. 75 (1987) 87–106.
[4] D. Angluin, C.H. Smith, Inductive inference: theory and methods, ACM Comput. Surv. 15 (1983) 237–269.
[5] J.L. Ba, J.R. Kiros, G.E. Hinton, Layer normalization, preprint, arXiv:1607.06450, 2016.
[6] F. Bacchus, C. Boutilier, A.J. Grove, Rewarding behaviors, in: Proceedings of the 13th National Conference on Artificial Intelligence (AAAI), 1996, pp. 1160–1167.
[7] A. Camacho, R. Toro Icarte, T.Q. Klassen, R. Valenzano, S.A. McIlraith, LTL and beyond: formal languages for reward function specification in reinforcement learning, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI), 2019, pp. 6065–6073.
[8] A.R. Cassandra, L.P. Kaelbling, M.L. Littman, Acting optimally in partially observable stochastic domains, in: Proceedings of the 12th National Conference on Artificial Intelligence (AAAI), 1994, pp. 1023–1028.
[9] J. Corazza, I. Gavran, D. Neider, Reinforcement learning with stochastic reward machines, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence (AAAI), 2022, pp. 6429–6436.
[10] G. De Giacomo, M. Favorito, L. Iocchi, F. Patrizi, A. Ronca, Temporal logic monitoring rewards via transducers, in: Proceedings of the 17th International Conference on Knowledge Representation and Reasoning (KR), 2020, pp. 860–870.
[11] P. Dhariwal, C. Hesse, O. Klimov, A. Nichol, M. Plappert, A. Radford, J. Schulman, S. Sidor, Y. Wu, P. Zhokhov, Openai baselines, https://github.com/openai/baselines, 2017.
[12] T. Dohmen, N. Topper, G. Atia, A. Beckus, A. Trivedi, A. Velasquez, Inferring probabilistic reward machines from non-Markovian reward signals for reinforcement learning, in: Proceedings of the 32nd International Conference on Automated Planning and Scheduling (ICAPS), 2022, pp. 574–582.
[13] F. Doshi-Velez, D. Pfau, F. Wood, N. Roy, Bayesian nonparametric methods for partially-observable reinforcement learning, IEEE Trans. Pattern Anal. Mach. Intell. 37 (2013) 394–407.
[14] G. Dulac-Arnold, N. Levine, D.J. Mankowitz, J. Li, C. Paduraru, S. Gowal, T. Hester, Challenges of real-world reinforcement learning: definitions, benchmarks and analysis, Mach. Learn. 110 (2021) 1–50.
[15] G. Dulac-Arnold, D. Mankowitz, T. Hester, Challenges of real-world reinforcement learning, CoRR, arXiv:1904.12901, 2019.
[16] D. Furelos-Blanco, M. Law, A. Russo, K. Broda, A. Jonsson, Induction of subgoal automata for reinforcement learning, in: Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI), 2020, pp. 3890–3897.
[17] M. Gaon, R. Brafman, Reinforcement learning with non-Markovian rewards, in: Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI), 2020, pp. 3980–3987.
[18] M. Ghavamzadeh, S. Mannor, J. Pineau, A. Tamar, et al., Bayesian reinforcement learning: a survey, Found. Trends Mach. Learn. 8 (2015) 359–483.
[19] G. Giantamidis, S. Tripakis, Learning Moore machines from input-output traces, in: Proceedings of the 21st International Symposium on Formal Methods (FM), 2016, pp. 291–309.
[20] F. Glover, M. Laguna, Tabu search, in: Handbook of Combinatorial Optimization, Springer, 1998, pp. 2093–2229.
[21] E.M. Gold, Complexity of automaton identification from given data, Inf. Control 37 (1978) 302–320.
[22] Gurobi Optimization, LLC, Gurobi optimizer reference manual, http://www.gurobi.com, 2018.
[23] M. Hasanbeig, N.Y. Jeppu, A. Abate, T. Melham, D. Kroening, Deepsynth: automata synthesis for automatic task segmentation in deep reinforcement learning, in: Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI), 2021, pp. 7647–7656.
[24] M. Hausknecht, P. Stone, Deep recurrent q-learning for partially observable MDPs, in: AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents (AAAI-SDMIA15), 2015.
[25] C. De la Higuera, Grammatical Inference: Learning Automata and Grammars, Cambridge University Press, 2010.
[26] C.C. Hung, T. Lillicrap, J. Abramson, Y. Wu, M. Mirza, F. Carnevale, A. Ahuja, G. Wayne, Optimizing agent behavior over long time scales by transporting value, CoRR, arXiv:1810.06721, 2018.
[27] IBM, ILOG CP Optimizer 12.8 Manual, 2018.
[28] M.T. Izadi, D. Precup, Using rewards for belief state updates in partially observable Markov decision processes, in: Proceedings of the 16th European Conference on Machine Learning (ECML), 2005, pp. 593–600.
[29] M. Jaderberg, V. Mnih, W.M. Czarnecki, T. Schaul, J.Z. Leibo, D. Silver, K. Kavukcuoglu, Reinforcement learning with unsupervised auxiliary tasks, CoRR, arXiv:1611.05397, 2016.
[30] N.Y. Jeppu, T. Melham, D. Kroening, J. O'Leary, Learning concise models from long execution traces, in: Proceedings of the 57th ACM/IEEE Design Automation Conference (DAC), IEEE, 2020, pp. 1–6.
[31] M. Jünger, T.M. Liebling, D. Naddef, G.L. Nemhauser, W.R. Pulleyblank, G. Reinelt, G. Rinaldi, L.A. Wolsey, 50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art, Springer Science & Business Media, 2009.
[32] L.P. Kaelbling, M.L. Littman, A.W. Moore, Reinforcement learning: a survey, J. Artif. Intell. Res. 4 (1996) 237–285.
[33] A. Khan, C. Zhang, N. Atanasov, K. Karydis, V. Kumar, D.D. Lee, Memory augmented control networks, CoRR, arXiv:1709.05706, 2017.
[34] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, in: Y. Bengio, Y. LeCun (Eds.), Proceedings of the 3rd International Conference on Learning Representations (ICLR), 2015, http://arxiv.org/abs/1412.6980.
[35] M.L. Littman, An optimization-based categorization of reinforcement learning environments, in: From Animals to Animats 2: Proceedings of the Second International Conference on Simulation of Adaptive Behavior, 1993, pp. 262–270.
[36] M.L. Littman, R.S. Sutton, S. Singh, Predictive representations of state, in: Proceedings of the 15th Conference on Advances in Neural Information Processing Systems (NIPS), 2002, pp. 1555–1561.
[37] M. Mahmud, Constructing states for reinforcement learning, in: Proceedings of the 27th International Conference on Machine Learning (ICML), 2010, pp. 727–734.
[38] F. Memarian, Z. Xu, B. Wu, M. Wen, U. Topcu, Active task-inference-guided deep inverse reinforcement learning, in: Proceedings of the 59th IEEE Conference on Decision and Control (CDC), 2020, pp. 1932–1938.
[39] N. Meuleau, L. Peshkin, K.E. Kim, L.P. Kaelbling, Learning finite-state controllers for partially observable environments, in: Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (UAI), 1999, pp. 427–436.
[40] V. Mnih, A.P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 1928–1937.
[41] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, Nature 518 (2015) 529–533.

[42] D. Neider, J.R. Gaglione, I. Gavran, U. Topcu, B. Wu, Z. Xu, Advice-guided reinforcement learning in a non-Markovian environment, in: Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI), 2021, pp. 9073–9080.
[43] J. Oh, V. Chockalingam, S. Singh, H. Lee, Control of memory, active perception, and action in minecraft, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 2790–2799.
[44] I. Osband, Y. Doron, M. Hessel, J. Aslanides, E. Sezener, A. Saraiva, K. McKinney, T. Lattimore, C. Szepesvari, S. Singh, et al., Behaviour suite for reinforcement learning, in: Proceedings of the 8th International Conference on Learning Representations (ICLR), 2020.
[45] L. Peshkin, N. Meuleau, L.P. Kaelbling, Learning policies with external memory, in: Proceedings of the 16th International Conference on Machine Learning (ICML), 1999, pp. 307–314.
[46] D. Pisinger, S. Ropke, Large neighborhood search, in: Handbook of Metaheuristics, Springer, 2010, pp. 399–419.
[47] P. Poupart, N. Vlassis, Model-based Bayesian reinforcement learning in partially observable domains, in: Proceedings of the 10th International Symposium on Artificial Intelligence and Mathematics (ISAIM), 2008, pp. 1–2.
[48] G. Rens, J.F. Raskin, R. Reynouad, G. Marra, Online learning of non-Markovian reward models, preprint, arXiv:2009.12600, 2020.
[49] F. Rossi, P. Van Beek, T. Walsh, Handbook of Constraint Programming, Elsevier, 2006.
[50] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, CoRR, arXiv:1707.06347, 2017.
[51] M. Shvo, A.C. Li, R. Toro Icarte, S.A. McIlraith, Interpretable sequence classification via discrete optimization, in: Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI), 2021, pp. 9647–9656.
[52] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al., Mastering the game of Go without human knowledge, Nature 550 (2017) 354.
[53] S.P. Singh, T. Jaakkola, M.I. Jordan, Learning without state-estimation in partially observable Markovian decision processes, in: Machine Learning Proceedings 1994, Elsevier, 1994, pp. 284–292.
[54] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.
[55] R. Toro Icarte, T.Q. Klassen, R. Valenzano, S.A. McIlraith, Using reward machines for high-level task specification and decomposition in reinforcement learning, in: Proceedings of the 35th International Conference on Machine Learning (ICML), 2018, pp. 2112–2121.
[56] R. Toro Icarte, T.Q. Klassen, R.A. Valenzano, S.A. McIlraith, Reward machines: exploiting reward function structure in reinforcement learning, J. Artif. Intell. Res. 73 (2022) 173–208, https://doi.org/10.1613/jair.1.12440.
[57] R. Toro Icarte, R. Valenzano, T.Q. Klassen, P. Christoffersen, A. Farahmand, S.A. McIlraith, The act of remembering: a study in partially observable reinforcement learning, CoRR, arXiv:2010.01753, 2020.
[58] R. Toro Icarte, E. Waldie, T.Q. Klassen, R. Valenzano, M.P. Castro, S.A. McIlraith, Learning reward machines for partially observable reinforcement learning, in: Proceedings of the 32nd Conference on Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 15497–15508.
[59] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-learning, in: Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI), 2016, pp. 2094–2100.
[60] Z. Wang, V. Bapst, N. Heess, V. Mnih, R. Munos, K. Kavukcuoglu, N. de Freitas, Sample efficient actor-critic with experience replay, CoRR, arXiv:1611.01224, 2016.
[61] C.J.C.H. Watkins, P. Dayan, Q-learning, Mach. Learn. 8 (1992) 279–292.
[62] Z. Xu, I. Gavran, Y. Ahmad, R. Majumdar, D. Neider, U. Topcu, B. Wu, Joint inference of reward machines and policies for reinforcement learning, in: Proceedings of the 30th International Conference on Automated Planning and Scheduling (ICAPS), 2020, pp. 590–598.
[63] Z. Xu, B. Wu, A. Ojha, D. Neider, U. Topcu, Active finite reward automaton inference and reinforcement learning using queries and counterexamples, in: Proceedings of the 2021 International Cross-Domain Conference for Machine Learning and Knowledge Extraction (CD-MAKE), Springer, 2021, pp. 115–135.
[64] Z. Zeng, R.M. Goodman, P. Smyth, Learning finite state machines with self-clustering recurrent networks, Neural Comput. 5 (1993) 976–990.
[65] M. Zhang, Z. McCarthy, C. Finn, S. Levine, P. Abbeel, Learning deep neural network policies with continuous memory states, in: Proceedings of the 2016 IEEE International Conference on Robotics and Automation (ICRA), 2016, pp. 520–527.